Ground Antenna Scheduling for Satellite Communication Using Quantum Annealing

Felipe Esteban Puente Cárdenas
Politecnico di Milano
Milan, Italy
felipeesteban.puente@mail.polimi.it

Leonardo Guglielmi
Politecnico di Milano
Milan, Italy
leonardo.guglielmi@mail.polimi.it

***Abstract*—Space telecommunications are an emerging means of communication across the globe, with rapid recent advances. Despite their potential, they still face several technological constraints, including the scarcity of ground infrastructure, short satellite visibility windows, and overcrowded antennas. In this work, we tackle the Multi-Satellite Multi-Ground-Antenna Scheduling Problem using a hybrid classical–quantum algorithm. Classical reductions and decomposition produce smaller subproblems, which are sampled using classical simulated annealing and reconstructed into a schedule. Bounded classical optimization provides reference objectives and bounds for evaluating the reconstructed schedules.**

***Index Terms*—Satellite scheduling, ground-station scheduling, quantum annealing, maximum-weight independent set, QUBO.**

I. Introduction

Modern space operations and space telecommunications depend on a finite network of ground stations to maintain contact with a growing population of orbiting satellites. When demand for contacts exceeds the available capacity, scheduling must determine which satellite each antenna serves during the available visibility windows. Depending on the application, every such assignment must respect a wide range of hard constraints and operational preferences, including mutual exclusivity (an antenna can serve at most one satellite at a time), minimum and maximum contact durations, antenna slew and reconfiguration times between successive contacts, mission priorities, operational blackouts, and interference avoidance.

In its general multi-resource form—multiple satellites, ground stations, and antennas—the problem is known as the Multi-Satellite Multi-Ground-Station Scheduling Problem (MMSP). Satellite range scheduling is NP-hard in the general case [1]. The solution space grows combinatorially with the number of scheduling decisions, and exact classical methods, such as branch-and-bound and mixed-integer linear programming, can become computationally expensive on large or densely constraint instances. Their practical performance nevertheless depend on the formulation and structure of the instance.

Although the MMSP shares structural features with other classical combinatorial archetypes, such as interval scheduling, bin packing, and weighted set covering, the problem combines temporal restrictions with competition for shared resources, that make it harder to solve [2]. Visibility windows are determined by orbital geometry and restrict when contacts can take place; a low-Earth-orbit satellite may remain visible from a station only for a few minutes. Antenna exclusivity and reconfiguration times further constraint the placement of contacts within these windows. When contact opportunities compete for the same resources, the scheduler must select a compatible subset according to their operational value [1].

The expansion of satellite constellations further motivates the development of effective ground-station scheduling methods [3], [4]. The increasing number of satellites and ground resources can produce more candidate contacts and more interactions between scheduling decisions. These factors motivate approaches that improve efficient and scalable ground-station scheduling, which is a critical challenge for the evolution of space telecommunications.

A. Project proposal

In this project, we investigate a scheduling framework design around Quadratic Unconstrained Binary Optimization (QUBO), a formulation supported by quantum annealing. We consider two classical modeling approaches: A Constraint Optmization Problem (COP) and a Maximum-Weight Independent Set (MWIS) formulation. Both their structures provide opportunities for classical reduction and decomposition before QUBO construction.

Before decomposition, both formulations apply an exact dominance reduction to candidate contacts of the same satellite. A candidate can be removed when another has at least its weight and no additional conflicts with contacts of other satellites. This reduction preserves the optimal objective and can reduce the size of the subproblems passed to the subsequent stages.

This approach is motivated by the resource and connectivity limitations of quantum annealing hardware. So, rather than attempting to map the full scheduling instance onto a processor directly, the framework first identifies independent components and applies additional decomposition to oversized components, taking into consideration the interactions across

their boundaries. The resulting subproblems are translated into QUBO instances, and their solutions are reconstructed into a schedule and checked again for constraint violations.

After this classical pre-processing, each subproblem is translated into a Quadratic Unconstrained Binary Optimization (QUBO) problem and solved via a D-Wave's simulated annealing sampler. In a similar way, the evaluation will asses the modeling, decomposition and reconstruction methods. Likewise, the logical variable and coupler limits used during decomposition will serve as experimental size constraints, rather than proof that the resulting subproblems can or cannot be embedded on a particular quantum processor.

The main experimental result is the schedule reconstructed before any MILP or LP reference evaluation. Classical reference solving is used separately to assess its quality and establish bounds, so improvements supplied by these solvers are not attributed to the annealing-based workflow.

B. Report structure

The remainder of the report is organized as follows. Section II reviews the related work on satellite scheduling and annealing-based optimization. Section III-A introduces adiabatic quantum computing, quantum annealing and QUBO, while Sections III-B and III-C introduce COP and MWIS, respectively. The subsequent sections define the scheduling problem, describe the proposed solution, and present the experimental methodology. Section IV will present a formal definition of the problem and Section V will explain in depth our proposal. Section VI will report the experiment conducted to assess our proposal performance, and finally Section VII will present the experimental results, followed by the discussion and the conclusions and directions for future work.

II. State of the art

Owing to its structural similarity to several classical combinatorial optimization archetypes, the MMSP has been tackled with a wide range of approaches. An early example is the work in [5], which applies mixed-integer programming and heuristic procedures to satellite scheduling. The NP-hard nature of the satellite range scheduling was established in [1]. The MMSP has also been formulated as a multiprocessor task scheduling problem with time-dependent resource requirements [6]. Metaheuristic methods have also been explored, for instance through heuristics inspired by graph coloring problem [7], or by applying genetic algorithms and simulated annealing, as reviewed in [8]. The problem has also been studied in a multi-objective setting on benchmark instances, approximating the Pareto front to explore trade-offs between competing scheduling criteria [9].

Constraint-based approaches have also been applied to satellite mission scheduling. In [10], a constraint-based model for fleets of Earth-observing satellites combines temporal and resource constraints with stochastic heuristic search. Although its scope includes observation planning, on-board storage, and downlink operations rather than only ground contacts, it provides a precedent for exploiting constraints during the construction of satellite schedules.

Another line of work formulates the problem as a continuous-time mixed-integer linear program and solves it through decomposition: particle swarm optimization handles the multi-station task assignment, while a separate optimization performs the scheduling within each single station, illustrating the use of hierarchical decomposition [11]. Machine learning approaches have been explored as well: in [12] deep reinforcement learning handles task assignment, while a heuristic method performs single-antenna scheduling with the objective of maximizing total reward. In the broader context of Earth-observation scheduling, multi-agent systems haver also been investigated to coordinate satellite operations and respond to changes in resource availability [13].

Overall, a recurring challenge in the literature is balancing solution quality and computational cost as scheduling instances grow in size and complexity. Moreover, despite the variety of methods proposed, many formulations share a combinatorial core: a constrained assignment problem whose solution space can grow exponentially with the number of scheduling decisions. Discrete formulations with binary decisions and pairwise constrains can be translated into QUBO form, motivating the investigation of quantum annealing (QA), which uses quantum fluctuations to search for low-energy solutions and has been studied on complex scheduling problems [14], [15]. However, practical performance depends on the formulation and experimental conditions, including the overhead of embedding the problem onto the quantum hardware [16].

III. Background

A. Adiabatic Quantum Computing and Quantum Annealing

1) The quantum adiabatic model: Adiabatic quantum computation (AQC) [17] is an alternative to the circuit-based (gate) model of quantum computation: rather than applying a sequence of discrete unitary gates to a register of qubits, AQC encodes a computation as the continuous-time evolution of a single system governed by a time-dependent Hamiltonian $\mathcal{H}(t)$. Despite this difference, the universal adiabatic model and the gate model have been shown to be polynomially equivalent: any circuit of L gates can be simulated by an adiabatic evolution with only polynomial overhead in L, and vice versa [18].

An AQC algorithm for an optimization problem P, with objective function $f(\mathbf{x})$ defined over n binary variables, is specified by three components:

- an *initial Hamiltonian* $\mathcal{H}_I$, chosen so that its ground state is easy to prepare and serves as the initial state of the system;
- a *final* (or *problem*) *Hamiltonian* $\mathcal{H}_F$, constructed so that its ground state encodes an optimal solution to P;
- an *adiabatic path* $s(t)$, a monotonically decreasing function with $s(0) = 1$ and $s(t_f) = 0$ for some total evolution time t_f.

These elements are combined into a single time-dependent Hamiltonian that shifts continuously from $\mathcal{H}_I$ to $\mathcal{H}_F$:

$$\mathcal{H}(t) = s(t)\,\mathcal{H}_I + \big(1 - s(t)\big)\,\mathcal{H}_F. \tag{1}$$

A register of n qubits is initialized in the ground state of $\mathcal{H}_I$ and evolved under (1). Under the conditions of the adiabatic theorem, sufficiently slow evolution keeps the system close to its instantaneous ground state, so that the final measurement yields an optimal solution of P with high probability.

The conditions under which this procedure is guaranteed to succeed are given by the **adiabatic theorem** [17]. The admissible speed of the evolution depends on by the *spectral gap*, defined as the energy difference between the ground state and the first excited state of the instantaneous Hamiltonian. When the gap is large, the ground state is well separated from the rest of the spectrum, and the evolution can generally proceed more quickly without inducing transitions to excited states. Conversely, when the gap narrows, transitions become harder to suppress, so the evolution may need to be slowed down accordingly. The minimum spectral gap $\delta_{\min}$ is therefore an important factor in the required evolution time. A commonly used estimate involves an inverse-square dependence on $\delta_{\min}$, although the adiabatic conditions also depend on the variation of the Hamiltonian and the chosen evolution schedule.

Consequently, the efficiency of an adiabatic algorithm depends on how $\delta_{\min}$ scales with the problem size: if the minimum gap decreases polynomially, the required evolution time remains polynomial; if it decreases exponentially, the evolution time grows exponentially as well.

2) Quantum annealing as a heuristic implementation: For NP-hard combinatorial optimization, satisfying the conditions required for an adiabatic guarantee can be difficult in practice, since $\delta_{\min}$ is generally unknown a priori and the required evolution time may be prohibitively long. Quantum annealing (QA) [19], [20], [21] uses quantum fluctuations to search for low-energy solutions and can be viewed as a heuristic approach when these guarantees are not enforced. A common QA formulation differs from the ideal adiabatic optimization procedure in the following respects:

- the final Hamiltonian is restricted to encode a classical objective function;
- the requirement that the system remain in the ground state throughout the evolution is relaxed: the final readout is used to obtain an approximate solution, and the probability of obtaining a satisfactory solution can be increased by repeatedly sampling the state of the system at the end of the evolution;
- the initial Hamiltonian, here called the *driver* Hamiltonian $\mathcal{H}_D$, is scaled by a tunneling coefficient $\Gamma(t)$ that is annealed from a large value to zero, yielding

$$\mathcal{H}(t) = \mathcal{H}_F + \Gamma(t)\,\mathcal{H}_D. \tag{2}$$

This construction is the direct quantum analogue of simulated annealing (SA) [22]: $\Gamma(t)$ plays the role of the SA temperature schedule $T(t)$, and quantum tunneling through energy barriers replaces thermally activated hops over them.

A convergence result analogous to the classical one underlying SA has also been established for QA: convergence to the ground state with probability approaching one as the number of iterations i grows is guaranteed provided that the tunneling coefficient decreases no faster than a power law, $\Gamma(i) \geq b/(i+1)^{c/n}$ for constants b, c, whereas SA requires a logarithmic cooling schedule, $T(i) \geq cn/\log i$. The QA bound is therefore more favourable in n, although both bounds are quite pessimistic [17]. Empirical studies based on numerical simulation have reported faster convergence for QA than for SA in nearly all cases compared, although this advantage is not universal and depends strongly on the structure of the energy landscape: SA performance degrades with the height of the barriers separating local minima, whereas QA performance degrades with their width.

3) QUBO model formulation: Optimization problems targeted by quantum annealing are commonly expressed in terms of the classical *Ising model*. Given n spin variables $s_i \in \{-1, +1\}$, external fields h_i, and pairwise couplings J_{ij}, the Ising energy function is

$$H(\mathbf{s}) = \sum_i h_i s_i + \sum_{i<j} J_{ij}\, s_i s_j. \tag{3}$$

Through the substitution $s_i = 1 - 2x_i$, the same problem can equivalently be cast, up to an additive constant, as a *Quadratic Unconstrained Binary Optimization* (QUBO) problem over binary variables $x_i \in \{0, 1\}$:

$$f_{\mathrm{QUBO}}(\mathbf{x}) = \sum_{i,j} Q_{ij}\, x_i x_j, \tag{4}$$

where linear terms are absorbed into the diagonal of Q, since $x_i^2 = x_i$. QUBO is the formulation most commonly targeted when mapping NP-hard problems onto quantum annealing hardware, as reductions from many classical combinatorial problems to QUBO are well established. Casting a scheduling problem in QUBO form typically requires expressing hard constraints (e.g., mutual exclusivity of resource assignments) as quadratic penalty terms added to the objective, so that infeasible assignments are pushed to high-energy states of $\mathcal{H}_F$. Preferences and soft constraints, in turn, are expressed as negative (rewarding) terms that lower the energy of the favored assignments.

4) Physical realization and the case for hybrid solvers: Since around 2007, quantum annealing has also been realized in purpose-built superconducting hardware, most notably by D-Wave Systems. These processors implement Eq. (2) using superconducting flux qubits, with h_i and J_{ij} constrained to match a fixed, sparse hardware connectivity graph. Early devices used the *Chimera* topology, with a qubit degree of at most 6; subsequent generations have substantially increased both qubit count and connectivity, moving through the *Pegasus* topology to the current *Zephyr* topology used by the Advantage2 system announced in 2025, which offers 20-way connectivity and over 4,400 qubits in D-Wave's Advantage2 system [23]. Since problem graphs rarely match the hardware graph, an arbitrary QUBO (or Ising) instance intended for

native execution generally requires *minor embedding* onto the hardware topology, a step that can require substantially more physical qubits than there are logical variables [24]. Combined with finite weight precision and residual environmental noise, this embedding overhead sharply limits the size and density of the problems that can be solved natively on annealing hardware.

This practical ceiling is precisely what motivates hybrid classical–quantum solvers, which use the quantum processor as a subroutine within a larger classical search procedure rather than attempting to solve an entire instance natively on the quantum hardware. Such hybrid architectures are the closest precedent, and the primary motivation, for the hybrid classical–quantum scheduling approach developed in this work.

B. Constraint Optimization Problem

A Constraint Optimization Problem (COP) extends a Constraint Satisfaction Problem (CSP) [22] with an objective function that ranks its feasible solutions. We first recall the CSP and then add the optimization layer.

A CSP is a triple $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$, where $\mathcal{X} = \{X_1, \dots, X_n\}$ is a set of variables, $\mathcal{D} = \{D_1, \dots, D_n\}$ their finite domains, with X_i taking values in D_i, and $\mathcal{C} = \{C_1, \dots, C_m\}$ a set of constraints. Each constraint $C_j = \langle S_j, R_j \rangle$ has a *scope* $S_j \subseteq \mathcal{X}$, the variables it involves, and a *relation* R_j listing their admissible combinations of values:

$$R_j \subseteq \prod_{X_i \in S_j} D_i. \tag{5}$$

Constraints are classified by their arity $|S_j|$: unary and binary constraints involve one or two variables, while *global* constraints, such as *Alldiff*, which forces all the involved variables to take distinct values, relate an arbitrary number of them. An assignment σ maps each variable X_i to a value $\sigma(X_i) \in D_i$ and is *consistent* if it satisfies every constraint,

$$\sigma|_{S_j} \in R_j \qquad \forall\, C_j \in \mathcal{C}, \tag{6}$$

where $\sigma|_{S_j}$ is the restriction of σ to S_j. Solving a CSP means finding a complete and consistent assignment; *map coloring* and *Sudoku* are classic examples. Deciding whether a finite-domain CSP has a solution is NP-complete in general [22], [25], although restricted classes, such as CSPs with a tree-structured constraint graph, can be solved in polynomial time [22].

A COP is a quadruple $\langle \mathcal{X}, \mathcal{D}, \mathcal{C}, F \rangle$, where $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$ is a CSP and $F : D_1 \times \cdots \times D_n \to \mathbb{R}$ scores every complete assignment. The constraints remain *hard* requirements, and the objective ranks the assignments that satisfy them. Solving a COP means finding

$$\sigma^* \in \arg \max_{\sigma \text{ consistent}} F(\sigma), \tag{7}$$

or, equivalently, minimizing a cost. The objective is usually *additive*, i.e., a sum of local functions on small scopes $T_k \subseteq \mathcal{X}$,

$$F(\sigma) = \sum_k f_k\big(\sigma|_{T_k}\big), \tag{8}$$

and in the simplest, *unary* case $F(\sigma) = \sum_i w_i\big(\sigma(X_i)\big)$, with a weight $w_i(a)$ for each value $a \in D_i$. Since its decision version contains the CSP, the COP is NP-hard in general.

Constraint propagation (or inference) prunes from the domains the values that cannot take part in any solution, either before or during the search. Its levels trade pruning power against computational cost: *node consistency* requires every value in a domain to satisfy the variable's unary constraints; *arc consistency* requires every value to have a compatible value in the domain of each neighboring variable; and *k-consistency* requires any consistent assignment to $k-1$ variables to extend consistently to any k-th variable. For a binary constraint R_{ik} between X_i and X_k, arc consistency of X_i with respect to X_k reads

$$\forall\, a \in D_i \;\; \exists\, b \in D_k : \; (a, b) \in R_{ik}, \tag{9}$$

and any $a \in D_i$ violating it can be removed, as it belongs to no consistent assignment; since only infeasible values are discarded, the optimum of a COP is preserved. A lighter form of inference applied during search is *forward checking*: after a variable is assigned, the values incompatible with that assignment are removed from the domains of its unassigned neighbors [22].

Optimization also allows *dominance*, which removes values that are feasible but never needed to reach the optimum. A value $a \in D_i$ is dominated by $b \in D_i$ if replacing a with b in any consistent assignment yields a consistent assignment with an objective at least as good. For a unary objective and binary constraints, a sufficient condition is

$$w_i(b) \geq w_i(a) \quad \text{and} \quad N_k(b) \subseteq N_k(a) \quad \forall\, X_k \neq X_i, \tag{10}$$

where $N_k(v) = \{\, u \in D_k : (v, u) \notin R_{ik} \,\}$ is the set of values of X_k incompatible with v. Removing a preserves the optimal value, although it may discard equivalent optimal solutions.

Finally, a constraint model may decompose into independent subproblems. If the variables can be partitioned into disjoint subsets $\mathcal{X} = \mathcal{X}_1 \cup \cdots \cup \mathcal{X}_p$ such that no constraint scope spans two of them, a solution to the CSP is the union of the solutions to the p subproblems. For a COP, the objective must also separate, $F(\sigma) = \sum_{q=1}^{p} F_q\big(\sigma|_{\mathcal{X}_q}\big)$, which holds for any additive objective whose scopes T_k do not span two subsets; the optimum is then the sum of the subproblem optima. We exploit this property later to reduce the size of the QUBO subproblems, although their suitability for embedding also depends on their connectivity on the target hardware.

C. Maximum-Weight Independent Set

The Maximum-Weight Independent Set (MWIS) problem is a classical combinatorial optimization problem defined on graphs. Given an undirected graph $G = (V, E)$, where V is the set of vertices and E the set of edges, and a weight function $w : V \to \mathbb{R}_{\geq 0}$ assigning a non-negative weight w_i to each vertex $i \in V$, an *independent set* is a subset of vertices $S \subseteq V$ such that no two vertices in S are connected by an edge. The MWIS problem consists of finding the independent set with maximum total weight $\sum_{i \in S} w_i$. When all weights

are equal and positive, the problem reduces to the Maximum Independent Set (MIS) problem, whose decision version is NP-complete through its equivalence to the clique problem on the complement graph [25]; consequently, MWIS is NP-hard in the general case.

The problem can be stated as a binary optimization problem by associating a binary variable $x_i \in \{0, 1\}$ with each vertex, where $x_i = 1$ indicates that vertex i is included in the set:

$$\max_{\mathbf{x}} \sum_{i \in V} w_i x_i \quad \text{s.t.} \quad x_i + x_j \leq 1 \quad \forall (i, j) \in E. \tag{11}$$

In this formulation, vertices represent candidate choices, weights express their value, and edges encode pairwise incompatibilities between choices. This makes MWIS a natural model for problems in which a set of mutually compatible options must be selected so as to maximize an additive reward. A further advantage of this formulation is that it translates directly into QUBO form: since all constraints are pairwise, each of them can be expressed as a quadratic penalty term without introducing auxiliary variables,

$$f_{\mathrm{MWIS}}(\mathbf{x}) = -\sum_{i \in V} w_i x_i + P \sum_{(i,j) \in E} x_i x_j, \tag{12}$$

where the penalty coefficient P can be chosen strictly greater than the largest vertex weight. This this choice, any infeasible assignment can be improved by removing a selected vertex involved in a conflict: the removed penalty exceeds the lost reward. Consequently, every global minimum of the QUBO corresponds to a maximum-weight independent set. Owing to this direct correspondence, independent-set problems have been studied through Ising and QUBO formulations [26].

MWIS also admits exact *dominance reductions*, analogous to the removal of dominated values in COP. Consider two adjacent vertices a and b, representing mutually exclusive choices. If $w(b) \geq w(a)$ and every vertex outside $\{a, b\}$ adjacent to b is also adjacent to a, then a can be removed: any independent set containing a remains independent, and has at least the same weight, when a is replaced by b. The reduction therefore preserves the optimal objective, although it may discard equivalent optimal solutions. Repeating such reductions can reveal additional dominances and change the connected-component structure of the remaining graph.

Finally, a graph may split into several connected components. Since an optimal independent set of a graph is the union of optimal independent sets of its components, each component defines an independent MWIS subproblem that can be solved separately and the global optimal objective is the sum of the component optima. This property complements the decomposition offered by the CSP-COP formulation and can reduce the size of the QUBO instances.

D. Exact Separator Decomposition

The independent-component decomposition discussed for COP and MWIS exploits the absence of interactions between separate parts of a problem. Separator decomposition extends this idea to connected problems by fixing a set of decisions whose removal separates the remaining connected components.

Many constraint and optimization problems can be represented by an *interaction graph*, whose vertices represent decision variables and whose edges connect variables that interact through a constraint or an objective term. For constraints involving several variables, all variables in the same scope are connected. This representation exposes structural properties that can be exploited during search, as studied in constraint-network decomposition and cutset conditioning [27].

A *vertex separator* is a set of vertices whose removal disconnects the interaction graph into smaller components. A single vertex with this property is called an *articulation point*. Unlike naturally disconnected components, the pieces obtained by removing a separator cannot generally be solved independently without accounting for the separator variables. Independence is obtained by *conditioning*: fixing an assignment to those variables and enforcing its consequences on the remaining problem [22], [27].

For each relevant separator assignment, the resulting components can be solved separately. In a satisfaction problem, their assignments are combined with the fixed separator assignment to obtain a complete solution. In an optimization problem with an additive objective, their optimal contributions are summed together with the contribution of the fixed variables, counting each term once. The best result is then selected across the separator assignments.

This procedure preserves the original problem when all relevant separator assignments are considered and all interactions are represented correctly. Obtaining a certified optimum additionally requires optimal solutions to the resulting optimization subproblems. The computational benefit therefore depends on the balance between the size of the separator, the number of assignments considered, and the difficulty of the remaining components.

E. Integer Programming and Linear Relaxations

Decomposition address the structure of a problem and its size, quantum annealing gives a tool to handle complex problems but it's intrinsict nature doesn't guarantee that the problem or the subproblem are going to be solved optimally. Integer programming and linear relaxations provide complementary tools for obtaining feasible solutions, bounding the optimal objective, and determining whether optimality has been reached.

A *Mixed-Integer Linear Program* (MILP) is an optimization problem with a linear objective and linear constraints, in which some or all decision variables are restricted to integer values. Binary variables are a special case, representing decisions such as selecting an option or assigning a resource. When all variables are required to be integer, the formulation is an Integer Linear Program (ILP). These models provide a general framework for expressing discrete decisions and operational constraints [28].

A *linear-programming relaxation* is obtained by removing the integrality requirements from a MILP problem while

retaining the linear constraints and variable bounds. Every feasible integer solution remains feasible in the relaxation, but additional fractional solutions become admissible. Consequently, for a maximization problem, the optimal relaxed objective is an upper bound on the integer optimum, while any feasible integer solution supplies a lower bound. For minimization, these bound directions are reversed.

Integer-programming algorithms exploit these bounds to guide the search. In *branch-and-bound*, the problem is divided into smaller subproblems, and branches are discarded when their bounds show that they cannot improve the best feasible solution found so far, called the *incumbent* [29]. If the search closes the gap between the incumbent and the global bound, optimality can be certified, subject to the solver's numerical tolerances.

The relaxation can be strengthened by adding *valid inequalities*: constraints satisfied by every feasible integer solution that exclude some fractional solutions. For binary models with pairwise-exclusive choices, *clique inequalities* express that at most one choice can be selected from a set of mutually incompatible options [30]. A triangle is the simplest clique for which this can strengthen the pairwise relaxation. Although the pairwise constraints permit the fractional assignment $x_i = x_j = x_k = 1/2$, mutual incompatibility requires

$$x_i + x_j + x_k \leq 1. \tag{13}$$

Adding such inequalities produces a *triangle-strengthened LP relaxation*. It can tighten the bound without excluding any feasible integer solution, but it does not generally eliminate all fractional solutions or certify the integer optimum. A fractional feasible solution alone also does not supply the relaxation bound: for maximization, an optimal LP value or another valid upper-bound certificate is required.

IV. Problem Definition and Formulation

A. Problem description

Ground station scheduling is an operational resource-management problem that arises whenever a network of Ground Stations (GS) must support a set of concurrent satellite missions. The core decision is to select contact opportunities and assign a specific antenna of a specific station to a specific satellite pass, while respecting a set of operational constraints. In the general case, visibility windows are fixed and non-negotiable, as they are dictated by orbital geometry; a satellite in Low Earth Orbit, for instance, typically remains in view of a station for only a few minutes. Moreover, the problem is typically *oversubscribed*: the demand for contacts exceeds the available antenna time, so that not all requests can be served and a selection must be made according to their operational value.

An instance of the MMSP is defined by the following elements:

- a set of satellites $\mathcal{S} = \{s_1, \ldots, s_N\}$, each characterized by a base priority p_{s_i};
- a set of ground stations $\mathcal{G}$, where each station $g \in \mathcal{G}$ hosts a set of antennas $\mathcal{A}_g$, whose number defines the capacity c_g of the station;
- a scheduling horizon $H = [0, T]$;
- for each satellite-antenna pair (s, a), a set of visibility windows $V_{s,a}$, i.e., the time intervals during which s is visible from a;
- for each GS g, a set of blackout intervals $\mathcal{B}_g$ during which the station is unavailable (e.g., due to maintenance or interference avoidance);
- for each satellite s, a minimum and a maximum contact duration, $d_s^{\min}$ and $d_s^{\max}$;
- for each antenna a, a setup time τ_a, i.e., the minimum separation required between two consecutive contacts on the same antenna to account for slewing and reconfiguration.

B. Candidate contacts

Rather than treating time as a continuous variable, discrete *candidate contacts* are generated within each visibility window. In the implemented model, candidates have a fixed duration, and their start times advance by a fixed discretization step. They therefore represent alternative placements of a contact within its parent visibility window, rather than a partition of the window.

A candidate contact is a tuple $c = (s_c, g_c, a_c, v_c, t_c^s, t_c^e, w_c)$, representing a contact between satellite s_c and antenna $a_c \in \mathcal{A}_{g_c}$ of GS g_c, originating from visibility window v_c, starting at t_c^s, ends at t_c^e, and having weight w_c. Contact intervals are treated as half-open intervals $[t_c^s, t_c^e)$, so that contacts meeting exactly at and endpoint do not overlap. A candidate contact is admissible only if:

1) it lies entirely within its parent visibility window: $[t_c^s, t_c^e) \subseteq [\alpha, \beta)$ for $v_c = [\alpha, \beta) \in V_{s_c, g_c}$;
2) it does not intersect any blackout interval $\in \mathcal{B}_{g_c}$;
3) its duration satisfies $d_{s_c}^{\min} \leq t_c^e - t_c^s \leq d_{s_c}^{\max}$.

The implementation considered here, a common fixed contact duration is used for all candidates, rather than selecting different durations within these bounds.

The weight w_c quantifies the operational value of a candidate contact. It is computed once for each parent visibility window, so that all candidate contacts originating from the same window share the same value, and is obtained as a scaled convex combination of the following normalized terms:

- the effective priority of the parent opportunity, derived from satellite priority and adjusted according to the maximum elevation of the pass;
- the duration of the visibility window d_V, normalized by the longest window duration in the parent-window input used by the weighting stage;
- the urgency level, a discrete score that is non-increasing with the start time of the window, so that windows occurring earlier in the scheduling horizon are favoured;
- the maximum elevation e_v reached during the pass, normalized by $90°$.

Denoting these normalized quantities respectively by $\hat{p}_v$, $\hat{d}_v$, $\hat{u}_v$, and $\hat{e}_v$, respectively, the weight is defined as:

$$w_c = W_{\max}\left(\lambda_p\,\hat{p}_{v_c} + \lambda_d\,\hat{d}_{v_c} + \lambda_u\,\hat{u}_{v_c} + \lambda_e\,\hat{e}_{v_c}\right), \quad \lambda_k \geq 0 \quad \forall k \in \{p,d,u,e\}, \qquad \sum_{k\in\{p,d,u,e\}} \lambda_k = 1. \tag{14}$$

Coefficients λ_k determine the relative importance of each term and $W_{\max}$ sets the scale of the weights. This weight is a synthetic opportunity-utility score rather than a measurement of downloaded data or actual mission demand.

We denote by $\mathcal{C}$ the set of all admissible candidate contacts. Since the three admissibility conditions listed above involve a single contact at a time, they are enforced during the construction of $\mathcal{C}$ and do not appear as explicit constraints in the optimization problem. Each candidate contact is then associated with a binary decision variable $x_c \in \{0,1\}$, where $x_c = 1$ indicates that c is included in the schedule.

C. Constraints

The remaining constraints involve pairs of candidate contacts. Two distinct candidate contacts $c, c' \in \mathcal{C}$ are said to be in *conflict* if they cannot both be scheduled, which occurs in the following cases:

- **Antenna exclusivity:** they use the same antenna ($a_c = a_{c'}$) and overlap or leave insufficient time for reconfiguration. For contacts ordered so that $t^s_c \leq t^s_{c'}$, a conflict occurs when $t^e_c + \tau_{a_c} > t^s_{c'}$. A separation exactly equal to the setup time is therefore allowed.
- **Satellite exclusivity:** they involve the same satellite ($s_c = s_{c'}$), since the implemented model permits at most one contact per satellite over the entire scheduling horizon. Such candidates are therefore mutually exclusive even when their time intervals do not overlap.
- **Parent-window exclusivity:** they originate from the same parent visibility window ($v_c = v_{c'}$). Such candidates represent alternative placements of one contact opportunity, so at most one of them can be selected, even if their time intervals do not overlap.

We denote by $\mathcal{K} \subseteq \mathcal{C} \times \mathcal{C}$ the set of conflicting pairs, each of which yields the constraint $x_c + x_{c'} \leq 1$.

D. Objective

The goal of the scheduling problem is to select the set of mutually compatible candidate contacts with the maximum total weight. This leads to the following binary optimization problem:

$$\begin{aligned} \max_{\mathbf{x}} \quad & \sum_{c\in\mathcal{C}} w_c\,x_c \\ \text{s.t.} \quad & x_c + x_{c'} \leq 1 \quad \forall (c,c') \in \mathcal{K}, \\ & x_c \in \{0,1\} \quad \forall c \in \mathcal{C}. \end{aligned} \tag{15}$$

A satellite may also remain unscheduled. Under these assumptions, the COP and MWIS formulations represent the same underlying feasible contact-selection problem and maximize the same total candidate-contact weight. Their different representations and solution procedures may nevertheless produce different schedules.

E. Assumptions

The following simplifying assumptions are adopted in this work:

- visibility windows are precomputed from orbital data and assumed to be deterministic;
- base satellite priorities are static, while the effective priority and weight are computed separately for each parent visibility window and remain fixed for all its candidate contacts;
- each contact uses a single antenna (no antenna arraying);
- each ground station is equipped with a single antenna;
- each satellite receives at most one contact over the entire scheduling horizon and may remain unscheduled;
- candidate contacts have a common fixed duration;
- blackout intervals are not included in the configured scenarios, so $\mathcal{B}_g = \varnothing$ for every station;
- data volume, on-board memory, weather effects and fairness among missions are not modeled.

V. Proposed Solution

This section describes the steps of our proposed hybrid classical–quantum approach, which combines classical reduction and decomposition with the annealing of terminal QUBO subproblems. The current implementation uses classical simulated annealing for the sampling stage. The numerical parameters and experimental protocol are specified in Section VI.

In Figure 1 is reported a schematic view of all the steps.

A. Classical decomposition: MWIS

In the MWIS formulation, the list of candidate contact slots produced in Section VI-A is converted into a *conflict graph* $G = (V, E)$. Each node $v \in V$ represents a contact slot and carries the slot weight $w(v)$. Each edge connects two slots that cannot both be scheduled, for one of two reasons:

- **Same antenna:** the two slots overlap in time, or the gap between them is shorter than the antenna's reconfiguration time.

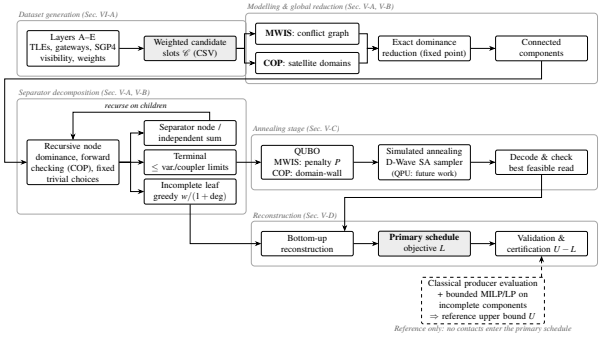


Fig. 1. Pipeline schema. Exact dominance reduction, connected component, and recursive dominance with separation are meant to work similarly with both formulation, although there are few changes

- **Same satellite:** each satellite is served at most once over the entire horizon, so all its slots are pairwise in conflict and form a clique. This also makes the candidate slots originating from the same visibility window mutually exclusive.

A feasible schedule is therefore an independent set of G, and the best schedule is a Maximum Weight Independent Set:

$$\max_{I \subseteq V} \sum_{v \in I} w(v) \quad \text{s.t. } \{u, v\} \notin E \; \forall u, v \in I, \; u \neq v. \tag{16}$$

The graph is reduced and decomposed through the following steps. Dominance reduction is applied before the initial connected-component extraction and repeated during recursive preprocessing.

1) **Exact dominance reduction.** A candidate contact a is dominated by another contact b of the same satellite when $w(b) \geq w(a)$ and every currently available contact of another satellite that conflicts with b also conflicts with a. Since the contacts of each satellite form a clique, an independent set containing a cannot contain another contact of that satellite. Replacing a with b therefore preserves feasibility and cannot decrease its weight, so a can be removed without changing the optimal objective. Equivalent alternatives with equal weights and identical external conflict sets retain the smallest slot identifier. The reduction is repeated until no further removal is possible, since removing contacts can reveal additional dominances.
2) **Connected components.** The reduced graph is split into its connected components. Since no edge connects different components, each component is an independent MWIS instance, and the global optimum is the sum of their optima. Dominance can change the component structure by removing vertices and their incident edges.
3) **Recursive preprocessing and trivial solutions.** At recursive nodes, dominance reduction is applied again to the remaining alternatives. Isolated vertices of the resulting graph are then selected directly, since their weights are non-negative and selecting them introduces no conflict. Their contribution is recorded separately. Independent residual pieces are processed separately while the decomposition budgets permit.
4) **Terminal subgraph encoding.** A remaining piece satisfying both logical size limits becomes a *terminal* subgraph. Each non-empty terminal is translated into a QUBO with one binary variable per retained slot, a reward proportional to its weight and a penalty on every edge, as described in Section V-C. A piece resolved entirely by exact preprocessing requires no sampling. Fitting terminals are accepted even when a depth or node budget prevents further decomposition.
5) **Exact separator decomposition.** A connected piece exceeding the terminal limits is decomposed recursively using separator conditioning. Single articulation points are preferred; otherwise, candidate pairs or small sets drawn from high-degree vertices are considered. Candidates are ranked by the size of the largest remaining piece and the balance of the split, with node identifiers used to break ties. For every feasible state of the chosen separator, i.e., every independent subset of its vertices, a branch is created. Selected separator vertices contribute their weights, their neighbours are excluded, and all separator vertices are removed from the residual problem. The remaining pieces are processed recursively, including the dominance and isolated-vertex reductions described above. Exploring every feasible separator state preserves the optimal objective, provided that the residual subproblems are solved optimally.
6) **Incomplete leaves and heuristic fallback.** Recursion is subject to limits on depth, decomposition nodes and separator states. An unresolved piece exceeding the terminal limits becomes an *incomplete* leaf when a budget prevents further decomposition or no separator is found within the configured search limits. It receives a deterministic greedy solution, obtained by visiting vertices in decreasing order of $w(v)/(1 + \deg(v))$ and accepting a vertex when it conflicts with none already selected. A conservative upper bound is obtained by summing the remaining vertex weights. Choices fixed during preprocessing are included in both the feasible objective and the bound, exactly once.

After annealing, terminal solutions are combined following the decomposition tree: independent pieces are summed, the best reconstructed branch of every separator is retained, and choices fixed during preprocessing are added exactly once. Dominated contacts are removed alternatives, rather than selected choices, and are not added back during reconstruction. Their removal preserves the optimal objective but may discard equivalent optimal schedules.

If no incomplete leaf occurs, the exact reductions and separator decomposition preserve the optimal objective of the original instance. Recovering that optimum nevertheless requires optimal terminal solutions, which annealing alone does not guarantee.

The same tree is also evaluated classically to obtain a baseline schedule and an upper bound for comparison. Small terminal graphs are solved by exhaustive enumeration or a maximum-weight clique solver on the complement graph, subject to configured size guards; larger terminals use the deterministic greedy heuristic and a conservative bound. The clique solver uses integer-scaled weights, which preserve the generated weights stored to three decimal places. At independent sums, bounds are added; at separator nodes, the bound is the maximum over all branch bounds, including branches not selected for the baseline schedule.

B. Classical decomposition: COP

Starting from the list of candidate contact slots produced in Section VI-A, we model the scheduling problem as a Constraint Optimization Problem. We then apply exact reductions and separator decomposition to obtain terminal subproblems

within the configured logical QUBO limits. Subproblems that cannot be decomposed within the available budgets are retained explicitly as incomplete leaves and are solved with greedy fallback heuristic method.

a) Model: Let S be the set of satellites and T_s the set of candidate slots of satellite $s \in S$. Every slot t is described by its antenna $a(t)$, start and end times $b(t) < e(t)$, antenna reconfiguration time $r(t)$ and weight $w(t)$ inherited from its parent visibility window. The COP has one variable x_s per satellite, with domain $D_s = T_s$. Each satellite therefore receives *at most one* contact over the entire horizon, by assigning one value to that variable. Two slots of different satellites *conflict*, written $t \sim t'$, when they use the same antenna and the later one starts before the earlier contact and its reconfiguration interval have ended:

$$t \sim t' \iff a(t) = a(t') \ \wedge \ e(t) + r(t) > b(t') \tag{17}$$

$$\text{with } b(t) \leq b(t') \tag{18}$$

For every pair of satellites $s \neq s'$ owning at least one conflicting pair of slots, a binary constraint $C_{s,s'}$ forbids assignments $(x_s, x_{s'}) = (t, t')$ with $t \sim t'$ representing temporal incompatibility. The objective is to find a consistent assignment maximizing the total weight:

$$\max_x \sum_{s \in S} w(x_s) \tag{19}$$

The satellites and constraints define the *constraint graph*, which drives the decomposition. Conflicts between slots of different satellites are computed once, comparing only slots on the same antenna, and shared by the derived subproblems. Reductions remove values from the current domains, and a constraint ceases to connect two variables when no conflicting pair remains in their domains.

b) Decomposition:

The COP is processed in the following steps.

1) **Dominance reduction.** A slot $a \in T_s$ is *dominated* by another slot $b \in T_s$ of the same satellite when $w(b) \geq w(a)$ and every currently available slot of another satellite conflicting with b also conflicts with a. Any schedule using a remains feasible and is not worse if a is replaced by b, so a can be removed without losing the optimum. Ties between slots with equal weights and identical conflict sets are broken by slot ID. Every removal shrinks the remaining conflict sets and may create new dominances, so the rule is applied repeatedly until a fixed point is reached.
2) **Connected components.** After dominance reduction, the constraint graph is split into connected components. Satellites in different components share no constraint, so each component is an independent subproblem and the optimum of the whole problem is the sum of their optima. No constraint is cut in this step.
3) **Recursive separator decomposition.** Each component is processed recursively using separator conditioning, as introduced in Section III-D. At every node, dominance reduction is applied again, we run *forward checking*, so slots conflicting with a chosen separator slot are removed from the other satellites, and satellites with no remaining constraint take their heaviest slot. Satellites with no remaining candidate slot are left unscheduled. Subject to the decomposition budgets, the reduced node then becomes one of the following:
   - an *independent sum*, when the reduced subproblem is disconnected: each piece becomes a child and the children's values are summed;
   - a *terminal*, when the QUBO of the chosen encoding satisfies the logical variable and coupler limits, or preprocessing resolves the subproblem entirely;
   - a *separator* node, when an affordable separator can be found: a small set K of satellites whose removal disconnects the constraint graph is selected, and one branch is created for every mutually consistent assignment of its satellites to a candidate slot or $\perp$.

   Candidate separators include articulation points, pairs of satellites for sufficiently small graphs, and subsets of up to $k_{\max}$ satellites drawn from a pool of high-degree vertices. They are ranked first by the number of slots in the largest remaining piece, then by the balance of the split and an estimate of the number of branches, with satellite name as tie-breaking criteria. The best-ranked candidate whose feasible assignments fit the remaining state budget is selected. Exploring every consistent separator assignment preserves the optimum when the residual subproblems are solved optimally. Identical subproblems reached through different branches are decomposed only once.
4) **Incomplete leaves and heuristic fallback.** The decomposition is bounded by limits on recursion depth, tree nodes and separator states. When a budget prevents further processing, or no affordable separator is found, the current node becomes an *incomplete* leaf. It receives a deterministic greedy schedule that visits slots in decreasing order of $w(t)/(1 + \deg(t))$ and accepts a slot when its satellite is still unscheduled and it conflicts with no accepted slot. Here, the degree counts conflicting slots of other satellites and alternative slots of the same satellite. The leaf also receives the upper bound $\sum_s \max_{t \in T_s} w(t)$, used as reference. Choices fixed during preprocessing are included in both the feasible objective and the bound.

c) Classical evaluation: Before annealing, every decomposition tree is evaluated bottom-up. COP terminals are solved exactly by branch and bound over the satellite assignments, using the sum of the heaviest remaining slots to prune branches that cannot improve the incumbent. Their configured size limits restrict the subproblems considered, but do not by themselves guarantee a short classical solution time. Independent sums add the values of their children, and separator nodes retain their best feasible branch. Upper bounds are combined by summing independent contributions and taking the maximum

over all separator branches. This yields a feasible *baseline schedule* and an upper bound on the global optimum..

d) From COP to QUBO: Each non-empty terminal is translated directly into a QUBO using the selected encoding of its satellite variables, as described in Section V-C. The default is domain-wall encoding [31], with one-hot encoding available as an alternative.

C. QUBO formulation and annealing

Both decompositions produce *terminal* subproblems whose logical QUBO sizes satisfy the configured variable and coupler limits. Each non-empty terminal is translated by its producer pipeline into a Quadratic Unconstrained Binary Optimisation problem:

$$\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^\top Q\mathbf{x}, \tag{20}$$

where Q is stored in upper-triangular form, with diagonal coefficients representing linear terms and off-diagonal coefficients representing quadratic interactions. The energy combines reward terms favouring valuable contacts with penalties discouraging incompatible assignments. The conditions needed to ensure that a ground state represents a feasible optimal schedule depend on the encoding.

a) MWIS terminals: Each slot v of a terminal subgraph is represented by a binary variable x_v, equal to one when the slot is scheduled. The QUBO is

$$H_{\mathrm{MWIS}}(\mathbf{x}) = -r\sum_{v\in V} w(v)x_v + P\sum_{\{u,v\}\in E} x_u x_v, \tag{21}$$

where $r > 0$ is a reward scaling factor and the penalty

$$P = p\big(r\max_{v\in V} w(v) + 1\big), \qquad p \geq 1,$$

is computed separately for each terminal. Since P exceeds every individual scaled reward, removing a selected vertex from a violated edge strictly lowers the energy. Consequently, a global minimum is a feasible maximum-weight independent set. Same-satellite cliques enforce the *at most one contact per satellite* condition alongside the antenna constraints.

b) COP terminals: Slot weights are normalised by the largest weight in the terminal, and each selected slot receives a reward proportional to its normalised weight. Antenna-conflict penalties discourage incompatible contacts. With the one-hot encoding, each slot is represented by a binary variable, and additional pairwise penalties forbid selecting two slots of the same satellite.

With the domain-wall encoding [31], used by default, a satellite with d candidate slots is represented by a chain of d bits $b_1, \ldots, b_d$, one per slot, with the slots ordered by antenna and start time. A valid chain is a block of 1s followed by a block of 0s, and the boundary between the two blocks, the *domain wall*, marks the chosen slot. The satellite takes slot k when b_k is the last 1 of the chain, and is unscheduled when all its bits are 0. For example, with $d = 4$, the chain 1100 selects the second slot, 1111 the fourth, and 0000 none. Formally, slot k is selected when the indicator

$$x_k = b_k - b_{k+1}, \qquad b_{d+1} = 0, \tag{22}$$

equals 1, i.e., when $b_k = 1$ and $b_{k+1} = 0$. A valid chain has at most one such position, so the constraint that each satellite takes at most one slot holds by construction. The only remaining requirement is that the chain be valid, i.e., that no 0 be followed by a 1. This is encouraged by a penalty $P\, b_k(1 - b_{k-1})$ on every pair of consecutive bits.

Compared with one-hot, which also uses one bit per slot, the advantage lies in the couplers within a satellite. The chain penalty only links consecutive bits, giving $d - 1$ couplers, whereas one-hot must penalize every pair of slots of the same satellite, giving $d(d-1)/2$. In one-hot, a conflict between two slots is a single product of two bits. In domain-wall, each indicator is the difference of two bits, so the conflict penalty

$$x_k\, y_j = (b_k - b_{k+1})(c_j - c_{j+1}) \tag{23}$$

expands into up to four couplers, where c denotes the chain of the other satellite. Some of these cancel when terms from different conflicts fall on the same pair of bits with opposite signs. Whether domain-wall yields fewer couplers overall therefore depends on the instance: it pays off when satellites have many candidate slots and relatively few conflicts. For this reason, the coupler count is computed from the resulting QUBO rather than estimated.

The default chain-penalty strength is determined from the conflict structure of each satellite. Its adequacy is assessed empirically rather than established by a general rule. In particular, a broken chain can produce negative slot indicators, so merely choosing a chain penalty larger than the maximum reward is insufficient to establish the same ground-state guarantee as for the MWIS encoding.

c) Annealing: The shared annealing stage reads the terminal QUBOs exported by either producer. Each QUBO is sampled independently with a configured number of reads and a seed determined by the base seed and terminal order. With fixed inputs, configuration and software environment, these choices support reproducible runs.

Every read is decoded into a set of selected slots and checked against the terminal's slot-level conflicts. The feasible read with the highest total (unnormalised) weight is retained, with energy used as a tie-breaking criterion. For domain-wall samples, this feasibility check concerns the decoded schedule; it does not separately certify that the original bit chain is valid. If no decoded read is feasible, the lowest-energy read is retained for diagnostics and the terminal is marked infeasible.

In the current implementation, sampling is performed with D-Wave's classical `SimulatedAnnealingSampler`. The stored QUBOs use Ocean's standard input format, which supports a future quantum sampling stage. Submission to physical hardware would additionally require minor embedding, coefficient-range checks and hardware-specific settings [24]. Satisfying the configured logical size limits does not establish that these requirements have been met.

D. Scheduling reconstruction

Terminal solutions are combined by traversing each component's decomposition tree bottom-up. The same combination rules are used for both formulations:

- a *terminal* contributes the slots selected by its retained annealing read, together with any choices fixed during preprocessing at that node;
- an *incomplete* leaf contributes its saved greedy solution, including the choices fixed at that leaf;
- an *independent sum* combines its children's schedules and adds their objective values, together with choices fixed at the current node;
- a *separator* node evaluates every branch using that branch's separator assignment and child solutions, adds choices fixed at the current node, and retains the branch of maximum reconstructed weight.

Fixed choices are included exactly once. Only the selected branch of each separator contributes to the final schedule; solutions from different separator states are not combined. The schedule of the whole instance is the union of the schedules of its independent components. A complete decomposition preserves the problem structure, but we remark that reconstructed schedule is certified optimal only when its feasible objective reaches a valid upper bound.

Contacts removed by dominance reduction are discarded alternatives, not choices fixed for inclusion in the schedule. They are therefore not added back during reconstruction. This applies both to the initial reduction and to reductions performed within separator branches. Since each removal preserves the optimal objective of the corresponding subproblem, reconstruction could use the retained alternatives while preserving the possibility of obtaining an optimal schedule for the original instance.

For the experimental analysis, the schedule obtained at this point is the *primary reconstructed schedule*. It combines sampled terminal solutions, choices fixed during preprocessing, separator assignments and greedy solutions of incomplete leaves. Its objective and provenance are evaluated before any MILP or LP reference solving.

a) Bounded MILP reference: A separate bounded classical reference evaluation examines components whose separator decomposition was incomplete in any branch. For both COP and MWIS, it operates on whole components retained after the initial exact dominance reduction, before recursive separator branching. This evaluates reference objectives and bounds without changing the primary schedule used in the main experimental comparison.

Each component's slot-level conflict graph is formulated as a binary integer linear program with edge and valid triangle-clique constraints. A proven optimum supplies a component reference objective; otherwise, usable feasible incumbents and valid bounds are recorded. A triangle-strengthened LP relaxation may additionally tighten the upper bound when it is solved successfully. The evaluation uses the time budgets specified in the experimental methodology.

The saved execution workflow also produces a post-MILP comparison schedule, replacing a component contribution when optimality is proved or a validated incumbent improves it. This schedule is reported separately from the primary reconstructed schedule. Since only separator-incomplete components are submitted to MILP, the procedure is not presented as a standalone MILP benchmark of the entire instance.

b) Validation and certification: The primary reconstructed schedule is checked against the saved slot-level conflict graph, including antenna incompatibilities and the at-most-one-contact condition for each satellite. Selected slot identifiers must correspond to unique rows of the original candidate-contact table, and the weight summed from those rows must agree with the reconstructed objective. Independent validation additionally checks the selected contacts against the original conflict graph before dominance reduction and verifies the recorded dominance witnesses; this confirms that removing alternatives has not introduced a violation of the original scheduling constraints. A schedule containing a conflict cannot be reported as a feasible solution.

Let L denote the objective of the primary reconstructed schedule. It is compared with a valid global upper bound U, obtained from the producer's component bounds and, where available, tightened by the classical reference evaluation. If $U - L$ is within the adopted numerical tolerance, the primary schedule is certified optimal; otherwise, $U - L$ bounds its remaining possible improvement. A post-MILP comparison schedule reaching the bound establishes a reference optimum, but does not certify the primary schedule unless its own objective also reaches that bound. Provenance is reported separately for the primary and post-MILP schedules.

VI. Experimental Methodology

The experimental campaign is designed to evaluate the feasibility, solution quality and computational cost of the proposed scheduling pipelines. Four experiments consider a baseline instance, satellite scaling, gateway scaling and sensitivity to the logical size limits imposed on terminal QUBOs. The scheduling assumptions, data-generation parameters and solver settings are held fixed except for the factors varied explicitly in each experiment.

A. Dataset generation

First we show the process to get the dataset used to run our experiments. The scheduling instances are produced by a five-layer data-wrangling pipeline. Each layer stores its result as an intermediate table, so that every stage can be inspected and re-run independently. Layers A and B have no mutual dependency and can therefore be executed independently, whereas Layers C combines their outputs and Layers D and E sequentially refine the resulting visibility windows. Orbital and ground-station data are drawn from public sources; the operational attributes introduced in Layer D are modelling assumptions rather than measured Starlink operating parameters due the lack of such information. Synthetic pseudo-random quantities are derived deterministically from fixed seeds. Reproducibility

also requires retaining the same orbital and ground-station input snapshots, since the public sources can change.

- **Layer A (satellite orbital data):** we obtain orbital elements of the Starlink constellation from CelesTrak's General Perturbations (GP) service in Two-Line Element (TLE) format [32]. Each record, consisting of a satellite name and two TLE lines, is parsed with the `skyfield` library [33], [34], which also provides the NORAD catalog identifier and the epoch of the element set. To keep the instance size under control, satellites are sorted by NORAD identifier and only the first N_{sat} are retained; the resulting subset is checked for missing identifiers, empty TLE lines and duplicate entries. This layer does not compute satellite positions: the element sets are propagated to the times of interest only in Layer C.
- **Layer B (ground stations):** we extract the locations of Starlink gateways, i.e. fixed earth stations connecting the constellation to the terrestrial network, from the public Hugging Face dataset `28hery/starlink-ground-stations` [35]. Since this dataset is unofficial and compiled from community sources, FCC filings and geocoding, it provides an approximation of the network rather than an actual reference. Rows with invalid coordinates or empty required fields are discarded and duplicate records with matching names and coordinates are merged. Each station is finally assigned an identifier determined by the sorted cleaned table. Both operational and planned locations are retained; treating them as available scheduling resources is therefore a modelling assumption.
- **Layer C (visibility windows):** we combine the two previous layers to compute the visibility windows of the selected satellite–station pairs. The scheduling horizon of length T is discretised into a uniform time grid with step Δt; at each grid point the satellite position is obtained by propagating its TLE with the SGP4 model [33]. A satellite is considered visible whenever its elevation angle $\varepsilon(t)$ with respect to some antenna is $\varepsilon(t) \geq \varepsilon_{\min}$, where $\varepsilon_{\min}$ is the adopted elevation mask. Maximal runs of consecutive visible samples are merged into windows described by their start and end times, duration and maximum sampled elevation; windows shorter than a minimum duration are discarded. Window boundaries are determined from the sampling grid, without refining the threshold crossings between samples.
- **Layer D (operational constraints):** we enrich each visibility window with the operational attributes of its ground station. Under our simplifying assumptions, as stated in IV-E, every gateway is modelled as a single-dish resource with capacity one, i.e. it can serve one contact at a time, and with a fixed reconfiguration time t_{rec} between consecutive contacts. The pipeline also supports synthetic blackout periods of configured length, placed deterministically within separate segments of the horizon. In the current configuration, the number of blackouts per station is zero (IV-E).
- **Layer E (candidate slots and weights):** we finally turn the available windows into the contact slot of the optimisation problem. Windows flagged as unavailable due conflicting with antenna's blackouts are discarded, and each remaining window generates fixed-length candidate slots of duration d_{c}, whose start times are spaced by a configured step δ; windows shorter than d_{c} yield no slot. Each slot links one satellite to one ground station and its antenna over a specific time interval. The weight defined in Eq. (14) is computed once per parent window and inherited unchanged by all its slots, so that alternative timings of the same pass are equally valuable and are discriminated through the conflict constraints.

In order to match the exact number of ground station and satellites for each experiment listed in VI-C, we used the script `trim_dataset.py` to extract from the whole dataset adequate instances. All configurations use the same frozen orbital and gateway input snapshot, archived on 27 September 2026. The orbital records in that snapshot have epochs between 16 and 18 June 2026; the common propagation horizon therefore starts at 00:00 UTC on 18 June 2026 and lasts four hours. The archive date is not the orbital-data epoch. Keeping propagation close to the element epochs limits the degradation associated with using older TLEs [34]. Satellites are selected deterministically by sorting the frozen satellite table by NORAD identifier and name. Gateways are selected by sorting the cleaned gateway table by station identifier and name. These prefixes are controlled test instances rather than random samples of the entire constellation or gateway network.

Visibility is sampled every 60 seconds using a uniform elevation mask of 25°. This mask is motivated by the SpaceX operating parameters discussed in FCC 21-48, which permits elevation angles as low as 25° for user terminals and gateways, with lower-angle exceptions for polar gateways [36]. We adopt 25° uniformly as a simplified experimental assumption. Visibility boundaries and maximum elevations are determined from the sampled grid.

Table I gives the common data-generation settings and the baseline subset sizes. They define a manageable scheduling model and are not claimed to be measured Starlink gateway operating parameters. Synthetic blackouts are disabled.

A window passing the 60-second visibility filter still generates no candidate contact unless it contains a complete four-minute interval. Both MWIS and COP formulations receive the same frozen weighted candidate-slot CSV for each scenario.

B. Synthetic priorities and contact weights

The weight function is the one defined in Eq. (14), with $W_{\max} = 100$ and coefficients

$$(\lambda_p, \lambda_d, \lambda_u, \lambda_e) = (0.45,\ 0.25,\ 0.20,\ 0.10).$$

These coefficients specify the project's synthetic utility model, giving the largest contribution to priority. They are not cal-

ibrated from operational mission data and are held fixed throughout all the experiments.

A satellite's base priority is an integer between 1 and 5, assigned deterministically from its NORAD identifier and the seed 20260531. For each parent visibility window, the priority is increased by one when its maximum sampled elevation is at least 60°, or decreased by one when it is below 20°, without exceding the 1–5 range. These thresholds are synthetic scoring choices. With the adopted 25° visibility mask, the below-20° adjustment is inactive, however we kept the the priority penalty in case of different value choices.

Urgency is also an integer between 1 and 5, computed for each parent window and inherited by its candidate slots. It decreases linearly with the window's start time within the scenario horizon $[T_0, T_1]$, where T_0 is the earliest window start rounded down to the minute and T_1 is the latest window end rounded up to the minute:

$$u = \mathrm{round}\left(1 + 4\left(1 - \frac{t_s - T_0}{T_1 - T_0}\right)\right). \tag{24}$$

Windows opening at the beginning of the horizon therefore receive higher urgency scores.

Priority and urgency are normalised as $(p-1)/4$ and $(u-1)/4$. Visibility-window duration is divided by the longest available parent-window duration in that scenario, and maximum elevation is divided by 90°. The resulting weight is inherited unchanged by every candidate slot of the parent window.

Because the duration normaliser and urgency interval are determined within each scenario, the set of experiments does not assume identical weights for every shared opportunity across different scenario sizes, in fact objective values are compared directly between solvers on the same frozen scenario.

C. Experimental design

The four experiments run are the following:

1) **Baseline.** Use 200 satellites and 50 gateways with the common data settings and terminal limits of 30 logical variables and 200 couplers. This experiment provides a reference for the pipeline's structural, feasibility, quality and runtime measurements.
2) **Satellite scaling.** Use 100, 200 and 500 satellites while keeping 50 gateways and all other settings fixed. This experiment examines how increasing the selected constellation subset affects candidate generation, conflicts, decomposition and computational cost.
3) **Gateway scaling.** Use 25, 50 and 100 gateways while keeping 200 satellites and all other settings fixed. This experiment examines how the selected ground-resource subset affects the instance structure and the schedules obtained.
4) **Terminal-size sensitivity.** Use the same frozen 200-satellite, 50-gateway instance with logical variable/coupler limits of 20/100, 30/200 and 50/350. This experiment examines how the terminal limits affect decomposition, sampling workload, incomplete leaves and the quality of the primary reconstructed schedule.

The baseline configuration is shared with all the experiments.

For the COP–MWIS comparison, both pipelines receive the same frozen candidate contacts and weights and enforce the same antenna constraints and at-most-one-contact-per-satellite assumption. They use the same logical terminal limits and configured decomposition budgets. Both apply exact dominance reduction before connected-component extraction and during recursive preprocessing. Their reduction procedures, encodings and classical baseline solvers remain those described in Section V. The experiments therefore compare the complete implemented pipelines rather than isolating the effect of the formulation or encoding alone.

D. Decomposition and solver settings

Table II reports the common decomposition settings and the classical MWIS solver guards. The state and node budgets apply separately to each component's decomposition. Exhausting a budget produces an explicitly incomplete leaf rather than an optimality assumption.

The terminal limits count logical binary variables and nonzero quadratic couplers. They are experimental controls and do not establish that a terminal has been embedded on physical quantum hardware. The exact-enumeration and exact-clique guards remain fixed at 22 and 50 vertices across the size-limit profiles, so the classical solver policy is not changed alongside the terminal limits.

TABLE I
COMMON DATA-WRANGLING PARAMETERS, WITH BASELINE SATELLITE AND GATEWAY COUNTS.

Layer	Parameter	Value
C	Baseline satellites / gateways	200 / 50
C	Horizon length T	4 h
C	Sampling step Δt	60 s
C	Elevation mask $\varepsilon_{\min}$	25°
C	Minimum retained visibility window	60 s
D	Antennas per station / capacity	1 / 1
D	Reconfiguration time t_{rec}	180 s
D	Blackouts per station	0
E	Candidate duration d_{c}	4 min
E	Candidate-start step δ	2 min

TABLE II
DECOMPOSITION AND CLASSICAL BASELINE SETTINGS. TERMINAL LIMITS SHOWN HERE ARE THE BASELINE VALUES.

Parameter	Value
Terminal logical-variable limit	30
Terminal coupler limit	200
Maximum separator size	3
High-degree candidate-pool size	14
Separator-state budget per component	10,000
Maximum recursion depth	12
Decomposition-node budget per component	10,000
MWIS enumeration vertex limit	22
MWIS complement-clique vertex limit	50

For MWIS QUBOs, the reward and penalty factors are both 1, giving the terminal penalty $P = \max_v w(v) + 1$. The COP pipeline uses domain-wall encoding by default, with maximum normalised assignment reward 5 and antenna-conflict penalty 10. Its chain penalties are computed separately for each satellite from the conflict structure, as described in Section V-C. If in COP the alternative one-hot encoding is used, its same-satellite penalty is 10.

Sampling is performed with D-Wave's classical `SimulatedAnnealingSampler` [37]. Each terminal receives 500 reads, with seed $42 + i$, where i is its zero-based position in the exported terminal list. The sampling call leaves the sweep count and temperature schedule at the library defaults: 1,000 sweeps and a geometric inverse-temperature schedule, with the temperature range determined from the problem coefficients [37]. The package version used it the 0.6.0 and should be retained with the final execution environment.

For each terminal, the highest-weight feasible decoded read is selected. Runs use a fixed seed schedule for reproducibility.

After reconstruction, incomplete components are examined separately through the bounded MILP reference evaluation described in Section V-D. The implementation uses `scipy.optimize.milp`, backed by HiGHS, with a requested relative MIP gap of zero [38]. MILP solves have a five-second per-component limit and a 30-second cumulative solver-call budget per experiment run. When MILP does not establish optimality, the triangle-LP relaxation can be solved through HiGHS with separate limits of 5 seconds per component and 30 seconds cumulatively. These are project budgets selected thorugh try outs during the development, and are not supported by actual litterature.

The MILP and LP budgets are independent and do not constitute a 30-second limit on the complete reference-evaluation stage. Graph preparation, triangle enumeration, validation and reporting occur outside the solver time limits. Solver statuses, usable incumbents, bounds and measured solver-call times are recorded. Objective and bound comparisons use an absolute tolerance of 10^{-6}.

E. Measurements and reproducibility

For each scenario and formulation, each experiment run records the following measurements:

- **Instance structure:** requested satellite and gateway counts, counts represented by candidate contacts, candidate-slot count, conflict or constraint count, and component sizes.
- **Reduction and decomposition:** candidate contacts removed by global dominance, retained contacts, component counts and sizes after the initial reduction, choices fixed during recursive preprocessing, generated terminal subproblems, terminal variable and coupler counts, separator states, recursion depth and incomplete leaves. Component counts after initial dominance are identified explicitly for both formulations. Global removals count distinct candidate contacts, while recursive removals are reported as occurrences across the recorded decomposition nodes.
- **Schedule quality:** selected-contact count, total unnormalised weight and conflict count of the primary reconstructed schedule, together with its gap to a valid reference bound or certified reference optimum.
- **Solver provenance:** contacts in the primary schedule supplied by sampled terminals, exact preprocessing, separator assignments and greedy incomplete leaves. Classical reference objectives, solver statuses and any post-MILP improvements are recorded separately.
- **Computational cost:** recorded times for modelling, reduction and decomposition, QUBO construction, classical producer evaluation, sampling and reconstruction. MILP/LP reference computation is identified separately wherever the saved timing records permit this separation.

The primary reconstructed schedule is the main experimental result. The classical producer baseline and the post-MILP comparison schedule are reported separately as references. Improvements introduced by MILP are excluded from the primary schedule's objective and provenance. Classical producer evaluation is also identified as a separate cost within the executed workflow. For the feasible primary reconstructed schedule with objective L and a valid reference upper bound U, the absolute gap bound is $U - L$. The relative gap bound is

$$g_{\mathrm{rel}} = 100\,\frac{U - L}{U}\,\%, \qquad U > 0, \tag{25}$$

with a value of zero when $U = 0$. When the reference evaluation certifies the optimal objective, this quantity measures the primary schedule's percentage shortfall from that optimum. Otherwise, it remains a bound on possible improvement, since U may be loose. Optimality of the primary schedule is reported only when its own feasible objective reaches a valid bound within the adopted tolerance. Recorded runtimes reflect the execution environments used for each formulation and are interpreted descriptively. Where a saved timing combines reconstruction and classical reference work, it is identified as a combined measurement rather than an isolated runtime of the primary pipeline. Subtracting MILP or LP solver-call time alone does not remove graph preparation, validation and reporting overhead. Input hashes, configuration copies, code revisions, software versions, terminal ordering and seeds are retained through the experiment run for reproducibility. Data-generation time is distinguished from optimization time when frozen candidate slots datasets are reused.

Table IV report the technical specification of the hardware on which the experiments have been conducted.

VII. Experimental Results

A. Overview and baseline experiment

The experiments compare the MWIS formulation with the COP formulation across the seven distinct experimental configurations shown in VI-C. Both approaches use the same candidate-contact dataset within each configuration. The primary result is the schedule reconstructed from sampled

TABLE III

PRIMARY SCHEDULE QUALITY (BEFORE MILP/LP REFERENCE EVALUATION). GAPS ARE MEASURED RELATIVE TO THE CERTIFIED REFERENCE OPTIMUM. ALL SCHEDULES ARE FEASIBLE; THE SHARED BASELINE IS LISTED ONCE. LIMITS DENOTE LOGICAL VARIABLES/COUPLERS. SLOTS ARE THE FINAL NUMBER OF SCHEDULED SLOTS; QUBOS ARE THE TERMINAL SUB-QUBOS EXPORTED TO THE ANNEALER.

Sat./GW; limits	MWIS weight	COP weight	MWIS gap (%)	COP gap (%)	MWIS slots	COP slots	MWIS QUBOs	COP QUBOs
100/50; 30/200	6,734.972	6,734.972	0.000	0.000	96	96	11	11
200/50; 30/200	12,576.412	12,429.057	0.040	1.211	180	180	1,286	176
500/50; 30/200	32,440.684	32,367.272	4.510	4.726	461	461	6	7
200/25; 30/200	12,585.515	12,601.387	2.400	2.277	184	185	36	306
200/100; 30/200	13,048.150	13,050.808	0.082	0.062	184	184	84	268
200/50; 20/100	12,575.915	12,428.405	0.044	1.216	180	180	1,022	166
200/50; 50/350	12,580.763	12,581.450	0.005	0.000	180	180	1,098	555

TABLE IV

HARDWARE AND SOFTWARE ENVIRONMENT USED FOR ALL EXPERIMENT RUNS.

Parameter	**Spec**
CPU	Intel i7-8565U CPU
RAM	8 GB DDR4
OS	Windows 11 Home 25H2
Python version	3.14.5

terminals, exact preprocessing choices, separator assignments and, where necessary, greedy solutions of incomplete leaves. MILP-generated contacts are excluded from these results; certified reference optima are used separately to evaluate schedule quality.

All fourteen primary schedules satisfy the scheduling constraints, including satellite exclusivity, antenna exclusivity and reconfiguration time. Table III reports their objective values and percentage gaps to the certified reference optima. Three schedules reach their reference optimum: both approaches in the 100-satellite case and COP with the largest terminal profile.

For the baseline configuration of 200 satellites, 50 gateways and terminal limits of 30 logical variables and 200 couplers, both approaches schedule 180 contacts. MWIS achieves an objective of 12,576.412, corresponding to a gap of approximately 0.040%, while COP achieves 12,429.057, with a gap of approximately 1.211%. Equal contact counts do not imply equal schedule quality, since the selected contacts have different operational weights.

Figure 2 distinguishes the contributions retained in the reconstructed schedules. In the *baseline experiment* sampled terminals contribute 80 contacts to MWIS and 50 to COP, representing approximately 44.4% and 27.8% of their respective schedules. MWIS completes the selected reconstruction without greedy contacts, whereas COP includes 37. These proportions describe from where each contact proportion came from, rather than their share in the objective or the causal benefit of sampling.

As Table III reports the *baseline experiment* exports 1,286 MWIS terminal QUBOs and 176 COP terminal QUBOs for sampling, but only 31 and 24 terminal solutions, respectively, are retained in the selected primary reconstruction. The remaining terminals belong to alternative separator branches. Consequently, the total sampling workload differs from the number of sampled subproblems that contribute to the final schedule. Similarly, incomplete leaves in discarded branches need not introduce greedy contacts into the selected reconstruction.

B. Satellite Scaling

Figure 3 shows how primary schedule quality changes across the 100-, 200- and 500-satellite scenarios, with 50 gateways and terminal limits of 30/200.

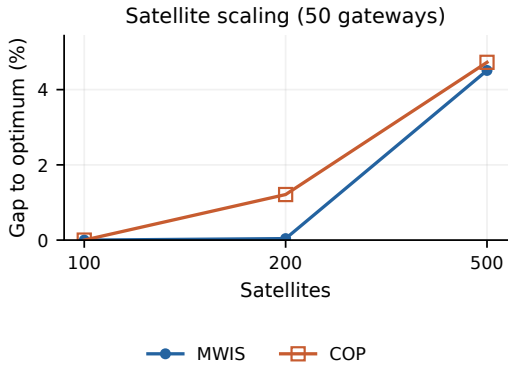


Fig. 3. Primary schedule gaps to the certified reference optima for satellite scaling, with 50 gateways and logical terminal limits of 30/200. Active resource subsets are not fully nested.

For 100 satellites, both formulations reach the certified optimum of 6,734.972 and schedule 96 contacts. Neither decomposition contains incomplete leaves. At 200 satellites, MWIS remains very close to the reference optimum, while COP exhibits a larger residual gap. At 500 satellites, both approaches experience a substantial deterioration in quality: the gaps increase to approximately 4.510% for MWIS and 4.726% for COP. Both schedules contain 461 contacts (Table III).

The reduced instance structure helps explain this change, as shown in Figure 4. After global dominance reduction, the 100-satellite graph contains 169 contacts, with a largest connected component of 20 contacts. The corresponding values increase to 460 and 315 in the 200-satellite case, and to 4,553 and 4,534 in the 500-satellite case. Almost the entire largest instance therefore belongs to one connected component.

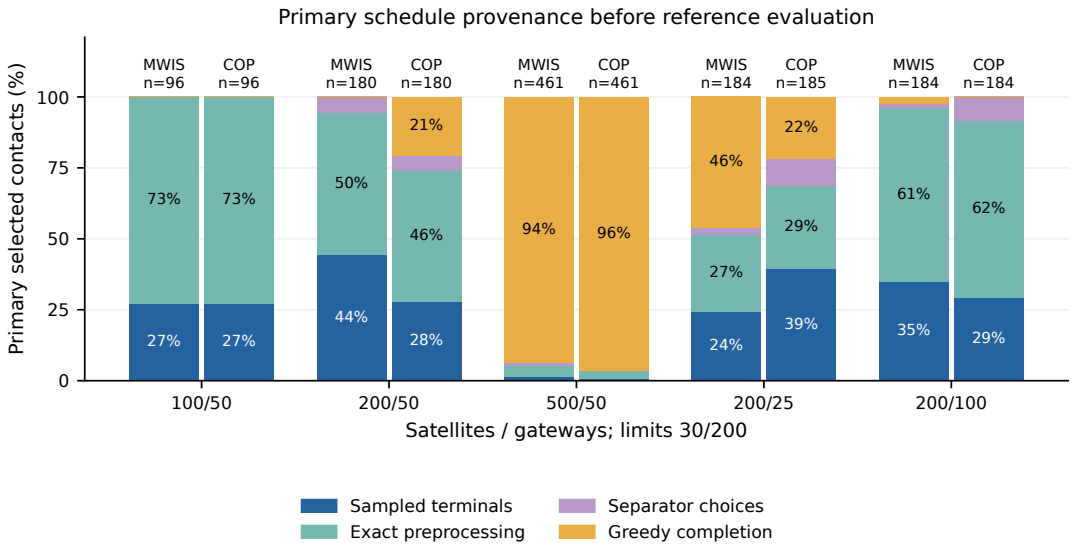


Fig. 2. Origins of contacts retained in the primary schedules before MILP reference evaluation. Shares describe contact provenance rather than objective contributions or computational effort.

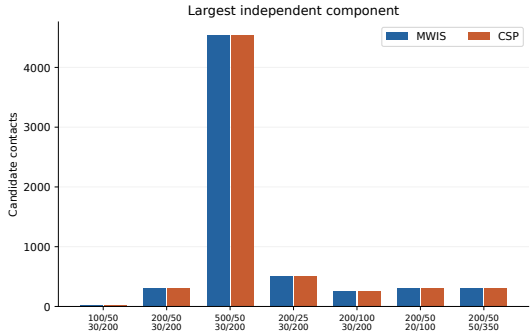


Fig. 4. Size of the largest connected component, in candidate slots, after global dominance reduction for each configuration

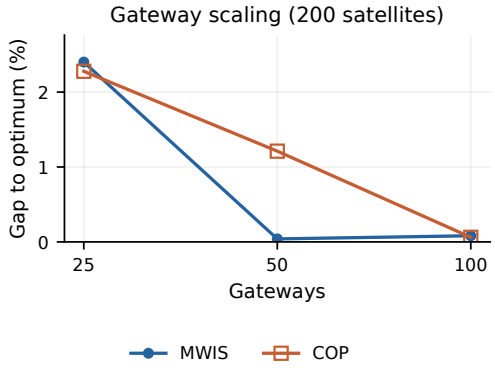


Fig. 5. Primary schedule gaps to the certified reference optima for gateway scaling, with 200 satellites and logical terminal limits of 30/200. Active resource subsets are not fully nested.

This structural concentration limits the effectiveness of bounded separator decomposition. Only six MWIS terminals and seven COP terminals reach sampling in the 500-satellite case. Figure 2 shows that greedy completion supplies 432(94%) of the 461 MWIS contacts and 444(96%) of the 461 COP contacts. The small sampling workload consequently reflects extensive unresolved work, rather than successful reduction of the full instance to a few small QUBOs.

Figure 11 illustrates this separation between small sampled terminals and large incomplete leaves. It also shows why average sampled-terminal size alone is insufficient to characterize decomposition success.

These comparisons describe the selected scenarios. Since their active resource subsets are not fully nested, the observed changes cannot be attributed exclusively to satellite count but also the different problem structure.

C. Gateway Scaling

Figure 5 compares 25, 50 and 100 gateways with 200 satellites and terminal limits of 30/200.

The 25-gateway scenario produces gaps of approximately 2.400% for MWIS and 2.277% for COP. With 50 gateways, these decrease to 0.040% and 1.211%, respectively. With 100 gateways, both approaches remain close to their reference optimum, with gaps of approximately 0.082% for MWIS and 0.062% for COP. The improvement is therefore substantial overall, although it is not strictly monotonic for MWIS.

The largest reduced connected component contains 503 contacts in the 25-gateway scenario, compared with 315 for 50 gateways and 264 for 100 gateways. The smaller components in the latter scenarios are consistent with improved decomposition and reduced dependence on greedy completion, as we

can see in Figure 2. In the 25-gateway case, greedy completion contributes 85 MWIS contacts and 40 COP contacts. In the 100-gateway case, these contributions fall to 4 and 0. However, preprocessing remains an important contributor in both formulations; this improvement in quality should not be interpreted as the result of sampling alone. As in satellite scaling, resource identities are not fully nested. These results demonstrate differences among the selected gateway scenarios rather than a general guarantee that adding gateways always produces the observed improvement.

D. Terminal-size Sensitivity

Terminal-size sensitivity uses the same frozen 200-satellite, 50-gateway instance for all three profiles: 20/100, 30/200 and 50/350 logical variables/couplers.

Figure 6 shows that MWIS remains close to the reference optimum throughout, with gaps of approximately 0.044%, 0.040% and 0.005%. COP changes little between the first two profiles, with gaps of approximately 1.216% and 1.211%, but reaches the certified optimum at 50/350. Larger terminal limits also reduce the number of incomplete leaves. MWIS records 1,648, 1,330 and 879 incomplete leaves across the three profiles, while COP records 153, 147 and 40. These counts include alternative branches that are subsequently discarded; they do not directly measure the contribution of incomplete leaves to the selected schedule.

Figure 12 demonstrates that annealing workload does not change monotonically with the limits. MWIS uses approximately 511,000, 643,000 and 549,000 reads, while COP uses 166,000, 176,000 and 555,000. Larger terminals can reduce further branching while allowing more previously unresolved pieces to reach sampling.

E. Sampling versus classical evaluation

To separate the loss caused by sampling from that caused by decomposition, each tree is also evaluated classically: same separator assignments and greedy leaves, but exact terminal solutions. With C the resulting objective, the primary loss splits as

$$\mathrm{OPT} - L = (\mathrm{OPT} - C) + (C - L), \tag{26}$$

where $\Delta = C - L$ is the sampling loss.

Table V shows $\Delta = 0$ in thirteen of the fourteen schedules; the exception is MWIS on the 50-logical variable 350-couplers instance, with $\Delta = 0.205$. The gaps of Table III, including the 500-satellite case, therefore stem almost entirely from decomposition and greedy completion. This does not prove that every sampled terminal is optimal, since terminals in discarded separator branches do not affect L, nor does it compare annealing with other heuristics at equal cost.

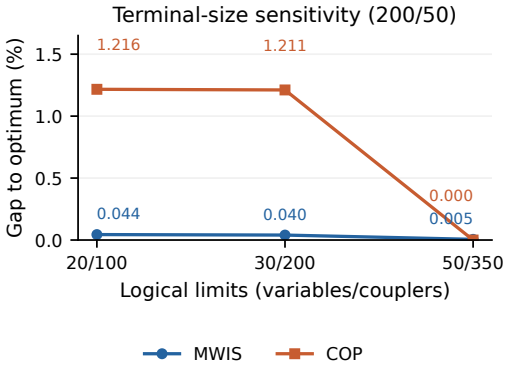


Fig. 6. Primary schedule quality for the three combined logical variable/coupler profiles, using the same frozen 200-satellite, 50-gateway instance.

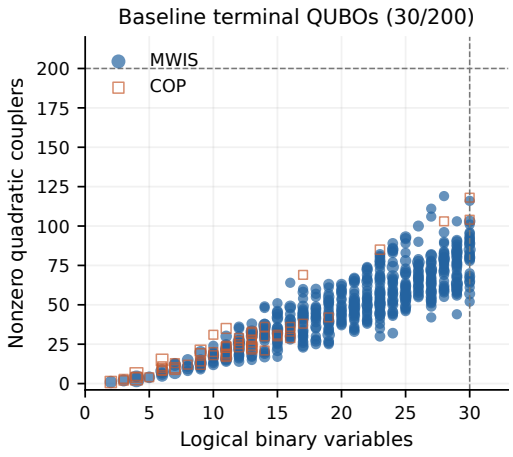


Fig. 7. Logical dimensions of the sampled baseline QUBOs. Dashed lines indicate the configured limits.

F. Logical dimensions and computational costs

Figure 7 shows that all sampled baseline terminals satisfy the configured logical limits. Across these terminals, MWIS averages approximately 18.42 binary variables and 44.47 couplers, compared with 8.02 variables and 16.44 couplers for COP. Figure 8 shows the average QUBO logical variables and couplers for each experiment run.

These averages concern the complete exported sampling workload, including alternative branches discarded during reconstruction. They reflect both decomposition and encoding decisions and should not be interpreted as a direct comparison of encoding efficiency alone. Moreover, they describe logical QUBO dimensions, before any physical embedding was performed.

Figure 9 shows the runtime exection of each phase of each experiment run. Figure 10 identifies which recorded classical stages account for the computational effort. In the MWIS baseline, classical producer evaluation dominates the profiled time, requiring approximately 2,422.9 seconds, compared with 88.9 seconds for separator decomposition. In the 500-satellite case, separator decomposition becomes the dominant stage for both approaches.

TABLE V

PRIMARY OBJECTIVE L (SIMULATED-ANNEALING TERMINALS) VERSUS THE CLASSICAL EVALUATION C OF THE SAME DECOMPOSITION TREE (EXACT TERMINAL SOLUTIONS, SAME GREEDY LEAVES). $\Delta = C - L$ ISOLATES THE LOSS ATTRIBUTABLE TO TERMINAL SAMPLING; THE CLASSICAL GAP $100\,(\mathrm{OPT} - C)/\mathrm{OPT}$ ISOLATES THE LOSS DUE TO DECOMPOSITION AND GREEDY COMPLETION.

	MWIS				COP			
Sat./GW; limits	L	C	Δ	Cl. gap (%)	L	C	Δ	Cl. gap (%)
100/50; 30/200	6,734.972	6,734.972	0.000	0.000	6,734.972	6,734.972	0.000	0.000
200/50; 30/200	12,576.412	12,576.412	0.000	0.040	12,429.057	12,429.057	0.000	1.211
500/50; 30/200	32,440.684	32,440.684	0.000	4.510	32,367.272	32,367.272	0.000	4.726
200/25; 30/200	12,585.515	12,585.515	0.000	2.400	12,601.387	12,601.387	0.000	2.277
200/100; 30/200	13,048.150	13,048.150	0.000	0.082	13,050.808	13,050.808	0.000	0.062
200/50; 20/100	12,575.915	12,575.915	0.000	0.044	12,428.405	12,428.405	0.000	1.216
200/50; 50/350	12,580.763	12,580.968	0.205	0.004	12,581.450	12,581.450	0.000	0.000

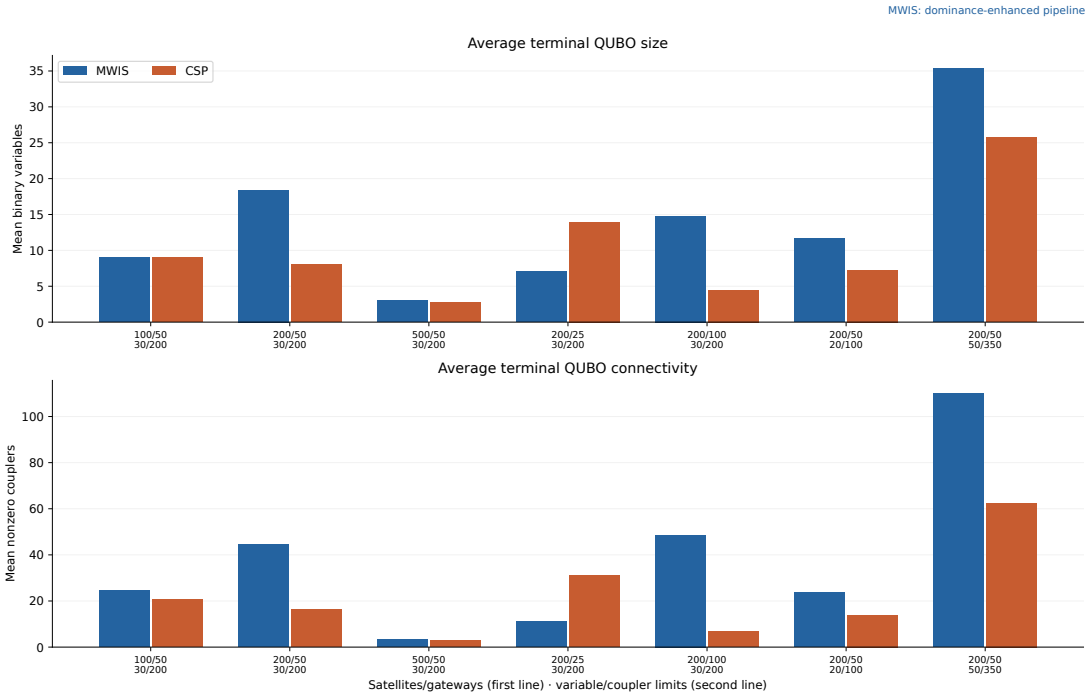


Fig. 8. Mean logical dimensions of the exported terminal QUBOs per configuration: binary variables (top) and nonzero quadratic couplers (bottom). Averages include terminals from separator branches discarded during reconstruction and reflect both decomposition and encoding choices, they do not isolate encoding efficiency.

Classical producer evaluation includes reference solutions of terminals and evaluation of incomplete leaves; it is distinct from annealing. However different execution environments or and shared timing blocks might change the runtime, so avoid to claim an overall speed superiority.

The sampling results demonstrate that the exported terminal QUBOs can be solved and reconstructed into feasible schedules using classical simulated annealing. They do not establish performance on quantum hardware or quantum advantage.

VIII. DISCUSSION

A. Viability of the hybrid scheduling approach

The experiments demonstrate that both implemented approaches can transform candidate-contact instances into bounded QUBO subproblems and reconstruct feasible schedules from their sampled solutions. All primary schedules satisfy the adopted scheduling constraints, and several configurations achieve objectives close to, or equal to, their certified reference optima. Given the reduced time required for the sampling phase with respecto to the others, this supports the viability of combining classical decomposition with an annealing-compatible optimization stage under the assumptions of this project.

The contribution of classical preprocessing is substantial: dominance reduction removes alternatives while preserving the optimal objective, and disconnected components can be processed independently. Separator conditioning extends this separation to connected instances, although its computational benefit depends on the number of assignments that must be explored and the sizes of the remaining pieces.

The largest satellite scenario exposes the limits of this approach. Its conflict graph after decomposition remains con-

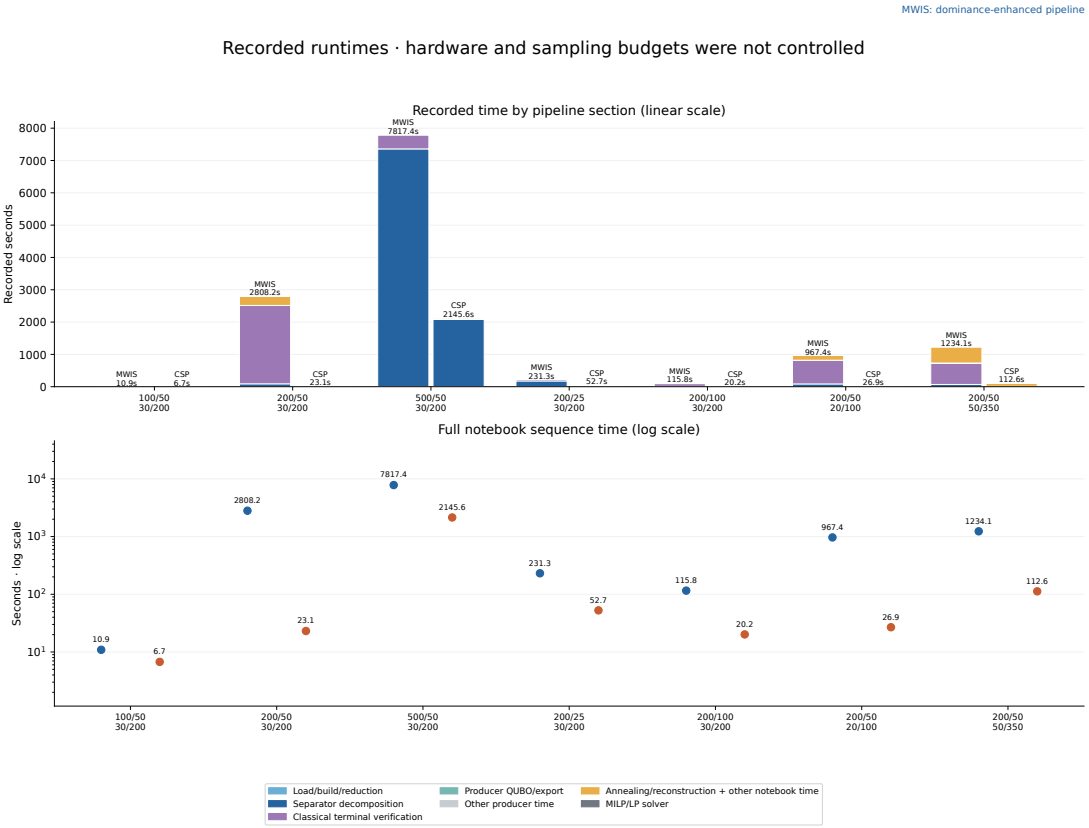


Fig. 9. Recorded runtimes per configuration: time by pipeline stage (top, linear scale) and full notebook execution time (bottom, log scale).

centrated in one large connected component, and bounded decomposition leaves most of the selected schedule to greedy completion. As illustrated by the provenance plot (Figure 2) and the supplementary leaf-size distributions, the existence of a few small sampled terminals does not imply that the complete instance has been successfully decomposed. Schedule quality and the fraction of the problem reached by sampling must therefore be considered together.

B. Differences between the MWIS and COP pipelines

The results do not identify one approach as uniformly superior. MWIS produces a smaller primary quality gap in the baseline and the smallest terminal profile, while COP reaches the certified optimum with the largest profile and performs slightly better in the tested gateway-scaling extremes. These differences indicate that the preferred implementation depends on both the instance structure and the computational limits.

Both pipelines retain the same weighted contact-conflict graph after global dominance reduction. Their subsequent processing nevertheless differs. MWIS operates directly on contact vertices, whereas COP groups contact alternatives into satellite domains and propagates separator assignments through those domains. The resulting search trees, branching workloads and terminal QUBOs can consequently differ even when the underlying scheduling choices are equivalent.

COP records fewer incomplete leaves in most tested configurations, but this does not consistently translate into a better selected schedule. Incomplete-leaf counts include alternative branches that may be discarded during reconstruction; a pipeline can therefore record many incomplete leaves while selecting a branch with little or no greedy contribution.

The comparison concerns the complete implemented pipelines. Differences in reductions, separator selection, encoding and classical evaluation prevent the observed outcomes from being attributed exclusively to the mathematical formulation. Similarly, the terminal-dimension differences shown in the logical QUBO plot (Figure 8) reflect both encoding and the particular subproblems generated by decomposition.

C. Contribution of sampling and the role of reference evaluation

The provenance measurements establish that sampled terminal solutions contribute directly to the primary schedules. Their contribution varies considerably across configurations: sampling supplies a meaningful fraction of the baseline schedules, although only a small fraction of the schedules in the 500-satellite scenario. This variation makes provenance an important complement to objective quality.

However, provenance alone does not demonstrate that sampling improves upon alternative classical methods, since a contact attributed to a sampled terminal may also be found

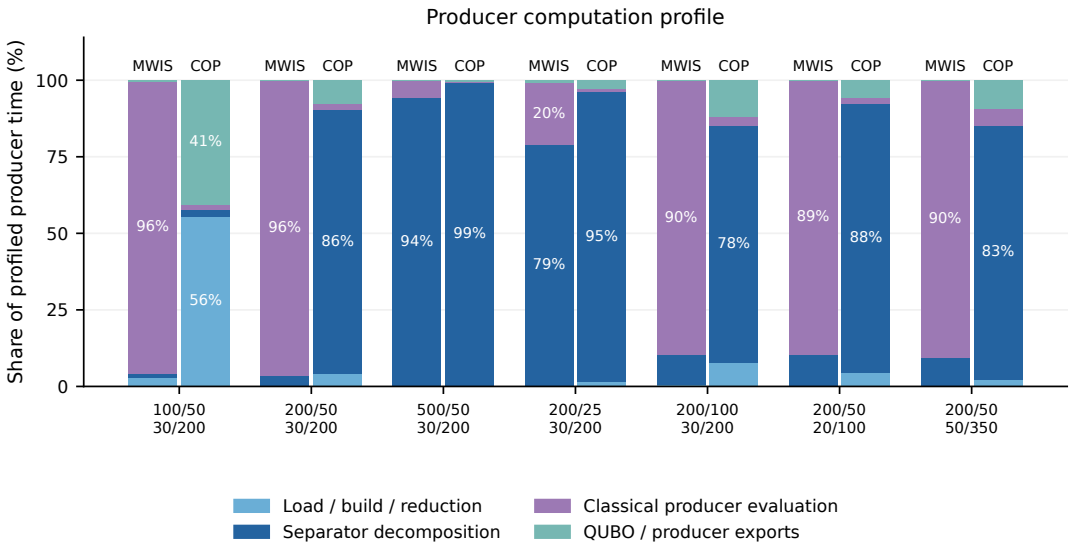


Fig. 10. Shares of the recorded classical producer stages. Classical evaluation is distinguished from sampling; the measurements do not represent complete primary-pipeline runtimes.

by an exact classical terminal solver. The classical evaluation of the same trees acts as a partial ablation (Table V): replacing sampled terminals with exact classical solutions changes the objective in only one of fourteen cases, and there the difference is small. Terminal sampling therefore introduces little observed objective loss relative to that reference. This does not establish global optimality when the trees contain incomplete leaves, because both evaluations depend on the same greedy completions and bounded decomposition. Nor does it compare annealing against other terminal heuristics at equal cost.

MILP provides a separate means of assessing the primary schedules. Its reference solutions and bounds reveal residual gaps that might otherwise remain hidden, particularly in incompletely decomposed components. Excluding MILP-generated contacts from the primary results preserves a clear distinction between the performance of the proposed workflow and subsequent classical reference evaluation. A reference schedule attaining the optimum therefore does not imply that the primary annealing-based reconstruction attained it.

D. Terminal limits and computational trade-offs

The terminal-size experiment shows that the logical limits affect the entire decomposition process, rather than only the size of the QUBOs submitted for sampling. Larger limits allow more pieces to become terminals, reduce some further branching and decrease the number of incomplete leaves. They can nevertheless increase the sampling workload by making previously unresolved subproblems eligible for sampling.

The improvement of COP at the largest profile illustrates the potential benefit of accepting larger terminals. MWIS also improves, although its primary schedule is already close to the reference optimum under the smaller profiles. Consequently, increasing terminal limits can have different practical value depending on the quality already achieved and the remaining unresolved structure.

Total reads do not provide a complete measure of computational efficiency. They depend on both the number of exported terminals and the per-terminal sampling budget, and they include alternative branches that do not contribute to the selected reconstruction. The substantial difference between sampled and retained terminals in the baseline highlights the cost of exploring competing separator assignments.

The computation profile also shows that classical processing can dominate the recorded workload. Exact terminal reference evaluation accounts for much of the MWIS baseline producer time, while separator decomposition dominates the largest satellite case. The usefulness of a hybrid approach therefore depends on controlling classical branching and evaluation costs as well as obtaining suitable terminal solutions.

The logical variable and coupler limits used here are experimental controls. They do not establish physical embeddability on a quantum processor, whose connectivity and embedding requirements would impose additional constraints.

E. Scope and limitations of the findings

Sampling was performed with classical simulated annealing. The experiments demonstrate the feasibility of the QUBO formulations and reconstruction procedures with this sampler, but they provide no direct evidence of quantum-hardware performance or quantum advantage.

Recorded timings can describe where computational effort occurs, but they cannot establish a controlled overall speed comparison between formulations. Moreover, some timing blocks combine reconstruction and classical reference work, limiting the separation of primary-pipeline costs.

Each method–configuration combination is represented by a single recorded run. Fixed inputs and seeds support repro-

ducibility, but they do not quantify variability across sampling seeds or independently generated instances. Small differences between approaches should therefore be interpreted as observations from this campaign rather than statistically established superiority.

The scaling scenarios additionally use resource subsets that are not fully nested. Their structural differences cannot be attributed solely to satellite or gateway counts. Terminal-size sensitivity provides a more controlled comparison because all profiles use the same frozen input.

Finally, the scheduling model assumes deterministic visibility, single-antenna gateways, synthetic contact weights and at most one contact per satellite over the scheduling horizon. These assumptions define the scope of the demonstrated feasibility and quality. More demanding operational models may change both the conflict structure and the effectiveness of decomposition.

IX. Conclusions and future work

A. Conclusions

The project investigated a hybrid approach to satellite ground-station scheduling that combined classical reduction and decomposition techniques with QUBO formulation and simulated annealing. Two different implementations were evaluated: a dominance-enhanced MWIS pipeline with standard one-hot encoding and a COP pipeline that uses dominance reduction and domain-wall QUBO encoding.

These two approaches produced feasible primary schedules in all the tested configurations. Several schedules reached, or closely approached, a certified optimal reference without using MILP-generated contacts. These results demonstrate that the implemented formulations, terminal sampling and reconstruction procedures can operate together consistently under the adopted scheduling assumptions.

The comparison results did not identify one pipeline as a clear superior. Their performance depended on the instance structure and terminal limits. Using larger terminal profiles improved schedule quality in the sensitivity experiment, while the largest satellite scenario exposed some limitations related to the bounded decomposition: most selected contacts originated from greedy completion, while only a small part of the instance actually reached sampling.

Therefore, the main outcome is a validated scheduling workflow whose quality, decomposition outcomes and solution provenance can be inspected separately. The use of classical reference evaluation provided an independent assessment of residual gaps. Despite the good results obtained, the sampling used classical simulated annealing, and no physical embedding was performed, so we cannot establish quantum advantage or performance on quantum hardware.

B. Future work

According to the obtained results, the main priority is to improve the decomposition process, specially on large densely-connected instances. Having a more effective separator selection, reuse of repeated subproblems and branch pruning based on valid bounds could reduce redundant work and increase the portion of the selected schedule supplied by sampled terminals.

The idea of parallelize the decomposition phase should be seriously analyzed in order to reduce the running time. These changes should analyze the possible effect on primary schedule quality, computational cost, and the reliability on greedy completion.

Another improvement could include a more controlled experimental comparison. The use of repeated runs with different sampling seeds and more input instances would better quantify variability. Additionally, using a separately timed reconstruction and reference evaluation would provide a clearer performance measurements.

Execution on quantum hardware would allow the annealing stage to be assessed directly. These experiments should measure the embedding requirements, resource usage, solution feasibility and its quality, and the complete time required to get a schedule. In addition, a comparison with classical terminal solvers would also be necessary in order to determine whether the quantum execution offers a real benefit.

Finally, the scheduling model could be extended to represent multiple antennas per station, repeated contacts with the same satellite, mission-specific communication requirements and realistic operational uncertainty. Repeated-contact models should preserve the identity of the satellite and enforce its communication constrains. Additionally, Data-volume objectives and changing operational conditions would also take the model a little closer to realistic scheduling needs.

APPENDIX A
SUPPLEMENTARY EXPERIMENTAL FIGURES

This appendix provides additional detail on decomposition outcomes, sampling workload. These figures support the analysis presented in the Experimental Results section.

A. Leaf-size distributions

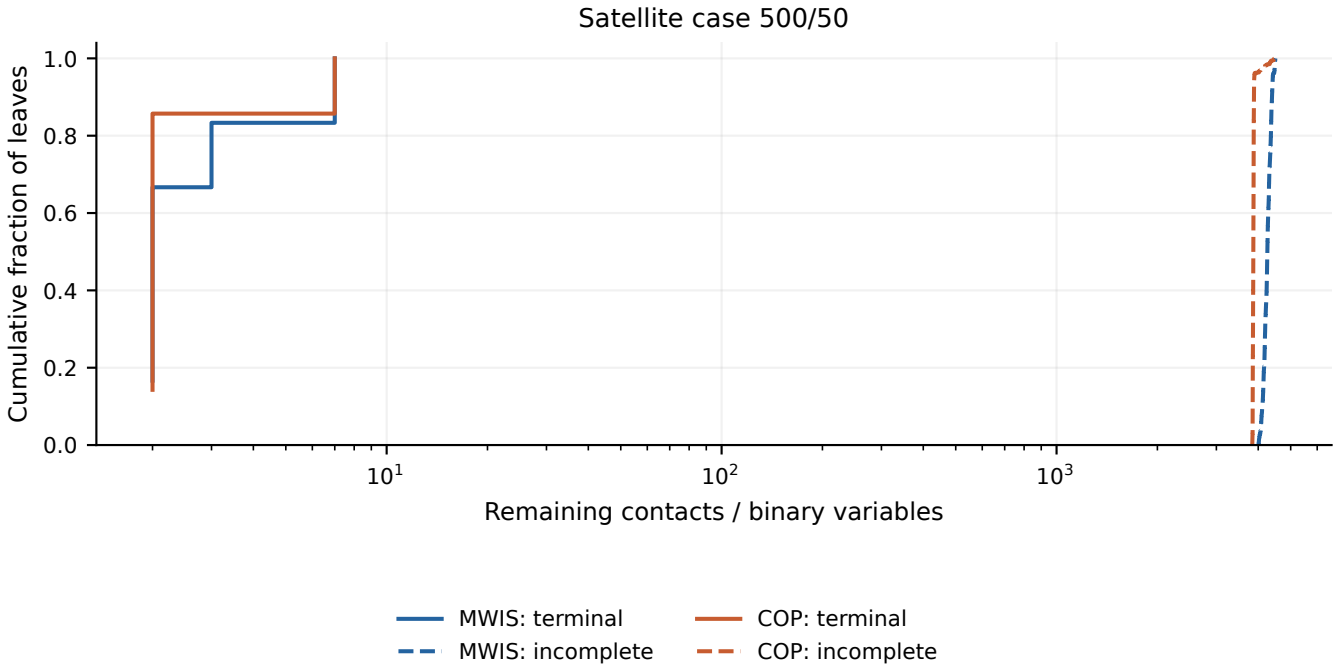


Fig. 11. Cumulative size distributions of terminal and incomplete leaves in the 500-satellite, 50-gateway scenario. Distributions include all recorded alternative branches.

B. Sampling workload

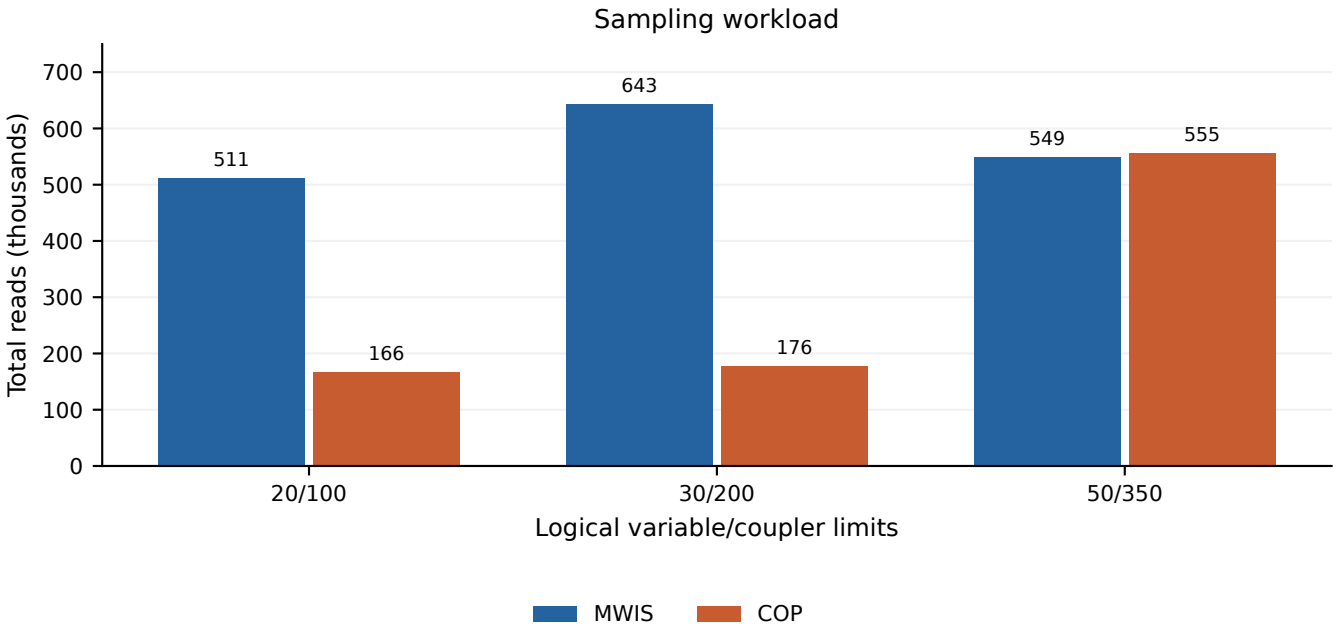


Fig. 12. Total sampling reads across all exported terminal branches. Per-terminal budgets are 500 reads for MWIS and 500 for COP; counts include alternatives discarded during reconstruction.

References

[1] W. L. Barbulescu L Watson JP, "Scheduling space–ground communications for the air force satellite control network," *Journal of Scheduling*, 2004. [Online]. Available: https://doi.org/10.1023/B:JOSH.0000013053.32600.3c.

[2] S. Dolan, S. Nayak, and H. Balakrishnan, "Satellite navigation and coordination with limited information sharing," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, N. Matni, M. Morari, and G. J. Pappas, Eds., ser. Proceedings of Machine Learning Research, vol. 211, PMLR, 15–16 Jun 2023, pp. 1058–1071. [Online]. Available: https://proceedings.mlr.press/v211/dolan23a.html.

[3] H. Krantz, E. C. Pearce, and A. Block, "The Steward Observatory LEO Satellite Photometric Survey," vol. 135, no. 1051, p. 095 003, Sep. 2023, Art. no. 095003. DOI: 10.1088/1538-3873/acf40c. arXiv: 2311.14092 [astro-ph.IM].

[4] M. Boley - A. C. - Byers, *Satellite mega-constellations create risks in low earth orbit, the atmosphere and on earth*, 2021. DOI: https://doi.org/10.1038/s41598-021-89909-7.

[5] T. D. Gooley, "Automating the satellite range scheduling process," M.S. thesis, Wright-Patterson Air Force Base, Ohio, Mar. 1993.

[6] F. Marinelli, S. Nocella, F. Rossi, and S. Smriglio, "A lagrangian heuristic for satellite range scheduling with resource constraints," *Computers & Operations Research*, vol. 38, pp. 1572–1583, Nov. 2011. DOI: 10.1016/j.cor.2011.01.016.

[7] N. Zufferey, P. Amstutz, and P. Giaccari, "Graph colouring approaches for a satellite range scheduling problem," *Journal of Scheduling*, vol. 11, Aug. 2008. DOI: 10.1007/s10951-008-0066-8.

[8] F. Xhafa and W. Ip, "Optimisation problems and resolution methods in satellite scheduling and space-craft operation: A survey," *Enterprise Information Systems*, vol. 15, pp. 1–24, Mar. 2019. DOI: 10.1080/17517575.2019.1593508.

[9] G. Petelin, M. Antoniou, and G. Papa, "Multi-objective approaches to ground station scheduling for optimization of communication with satellites," *Optimization and Engineering*, vol. 24, Mar. 2021. DOI: 10.1007/s11081-021-09617-z.

[10] J. Frank, A. Jonsson, R. Morris, and D. E. Smith, *Planning and scheduling for fleets of Earth observing satellites*, NASA Technical Reports Server, Preprint, document ID 20010087327, 2001. [Online]. Available: https://ntrs.nasa.gov/citations/20010087327.

[11] M. Tian, G. Ma, P. Huang, B. Cheng, and W. Li, "Optimizing satellite ground station facilities scheduling for rsgs: A novel model and algorithm," *International Journal of Digital Earth*, vol. 16, pp. 3949–3972, Sep. 2023. DOI: 10.1080/17538947.2023.2259870.

[12] J. Ou et al., "Deep reinforcement learning method for satellite range scheduling problem," *Swarm and Evolutionary Computation*, vol. 77, p. 101 233, Mar. 2023. DOI: 10.1016/j.swevo.2023.101233.

[13] P. Skobelev, E. Simonova, A. Zhilyaev, and V. Travin, "Application of multi-agent technology in the scheduling system of swarm of earth remote sensing satellites," *Procedia Computer Science*, vol. 103, pp. 396–402, 2017, XII International Symposium Intelligent Systems 2016, INTELS 2016, 5-7 October 2016, Moscow, Russia, ISSN: 1877-0509. DOI: https://doi.org/10.1016/j.procs.2017.01.127. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S187705091730128X.

[14] L. F. Pérez Armas, S. Creemers, and S. Deleplanque, "Solving the resource constrained project scheduling problem with quantum annealing," *Scientific Reports*, vol. 14, no. 1, p. 16 784, Jul. 2024. DOI: 10.1038/s41598-024-67168-6.

[15] W. Bożejko et al., "Optimal solving of a scheduling problem using quantum annealing metaheuristics on the d-wave quantum solver," *TechRxiv*, vol. 2023, no. 0512, 2023. DOI: 10.36227/techrxiv.22677721.v2. [Online]. Available: https://www.techrxiv.org/doi/abs/10.36227/techrxiv.22677721.v2.

[16] C. Carugno, M. Ferrari Dacrema, and P. Cremonesi, "Evaluating the job shop scheduling problem on a D-Wave quantum annealer," *Scientific Reports*, vol. 12, no. 1, p. 6539, Apr. 2022. DOI: 10.1038/s41598-022-10169-0.

[17] C. C. McGeoch, *Adiabatic Quantum Computation and Quantum Annealing*. Springer Cham, 2014. DOI: https://doi.org/10.1007/978-3-031-02518-1.

[18] D. Aharonov, W. van Dam, J. Kempe, Z. Landau, S. Lloyd, and O. Regev, "Adiabatic quantum computation is equivalent to standard quantum computation," *SIAM Journal on Computing*, vol. 37, no. 1, pp. 166–194, 2007. DOI: 10.1137/S0097539705447323.

[19] B. Apolloni, C. Carvalho, and D. de Falco, "Quantum stochastic optimization," *Stochastic Processes and their Applications*, vol. 33, pp. 233–244, 1989. DOI: 10.1016/0304-4149(89)90040-9.

[20] A. B. Finnila, M. A. Gomez, C. Sebnik, C. Stenson, and J. D. Doll, "Quantum annealing: A new method for minimizing multidimensional functions," *Chemical Physics Letters*, vol. 219, pp. 343–348, 1994. DOI: 10.1016/0009-2614(94)00117-0.

[21] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Physical Review E*, vol. 58, pp. 5355–5363, 1998. DOI: 10.1103/PhysRevE.58.5355.

[22] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. 2010.

[23] D-Wave Quantum Inc., *D-Wave announces general availability of Advantage2 quantum computer*, https://www.dwavequantum.com/company/newsroom/press-release/d-wave-announces-general-availability-of-

advantage2 - quantum - computer - its - most - advanced - and-performant-system/, May 2025.

[24] D.-W. Q. Inc., *D-wave minor embedding guide*. [Online]. Available: https://docs.dwavequantum.com/en/latest/quantum_research/embedding_intro.html.

[25] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds., Plenum Press, 1972, pp. 85–103.

[26] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014. DOI: 10.3389/fphy.2014.00005.

[27] R. Dechter, "Enhancement schemes for constraint processing: Backjumping, learning, and cutset decomposition," *Artificial Intelligence*, vol. 41, no. 3, pp. 273–312, 1990. DOI: 10.1016/0004-3702(90)90046-3. [Online]. Available: https://ics.uci.edu/~dechter/publications/r8.pdf.

[28] L. A. Wolsey, *Integer Programming*, 2nd ed. John Wiley & Sons, 2021. DOI: 10.1002/9781119606475.

[29] A. H. Land and A. G. Doig, "An automatic method of solving discrete programming problems," *Econometrica*, vol. 28, no. 3, pp. 497–520, 1960. DOI: 10.2307/1910129.

[30] M. W. Padberg, "On the facial structure of set packing polyhedra," *Mathematical Programming*, vol. 5, pp. 199–215, 1973. DOI: 10.1007/BF01580121.

[31] N. Chancellor, "Domain wall encoding of discrete variables for quantum annealing and QAOA," *Quantum Science and Technology*, vol. 4, no. 4, p. 045 004, 2019. DOI: 10.1088/2058-9565/ab33c2. [Online]. Available: https://arxiv.org/abs/1903.05068.

[32] T. S. Kelso, *A new way to obtain GP data*, CelesTrak, Updated June 23, 2026. Accessed October 9, 2026, 2020. [Online]. Available: https://celestrak.org/NORAD/documentation/gp-data-formats.php.

[33] D. A. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, "Revisiting spacetrack report #3," in *AIAA/AAS Astrodynamics Specialist Conference and Exhibit*, American Institute of Aeronautics and Astronautics, 2006. DOI: 10.2514/6.2006-6753. [Online]. Available: https://celestrak.org/publications/AIAA/2006-6753/.

[34] Skyfield Developers, *Skyfield documentation: Earth satellites*, Software documentation, Accessed October 10, 2026. [Online]. Available: https://rhodesmill.org/skyfield/api-satellites.html.

[35] juliensimon, *Starlink ground stations & points of presence*, Hugging Face dataset, 28hery mirror, Gateways configuration. Accessed October 9, 2026, 2026. [Online]. Available: https://huggingface.co/datasets/28hery/starlink-ground-stations.

[36] Federal Communications Commission, "Space exploration holdings, llc: Request for modification of the authorization for the SpaceX NGSO satellite system," Federal Communications Commission, Order and Authorization and Order on Reconsideration FCC 21-48, 2021. [Online]. Available: https://docs.fcc.gov/public/attachments/FCC-21-48A1.pdf.

[37] D-Wave Quantum, *SimulatedAnnealingSampler: Sampling parameters*, D-Wave Quantum Computing Products documentation, Accessed October 10, 2026. [Online]. Available: https://docs.dwavequantum.com/en/latest/ocean/api_ref_samplers/generated/dwave.samplers.SimulatedAnnealingSampler.sample.html.

[38] SciPy Developers, *scipy.optimize.milp: Mixed-integer linear programming*, SciPy documentation, Accessed October 10, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html.